When the Interval Is Smaller Than the Instrument

Two Ways Streaming Latency Benchmarks Fail on Sub-Millisecond Paths, and What They Left of a Kafka-versus-Redis Comparison

GUSTAVO PEDRO RICOU, School of Computer Science and Statistics, Trinity College Dublin, Ireland

Streaming brokers are compared on latencies of tenths of a millisecond using instruments whose own timescales are larger than that. We report two consequences, both of which leave the benchmark's output looking healthy, and both of which are governed by the same ratio: the instrument's timescale divided by the interval being measured.

The first is that the measurement can violate causality. Broker delay is a difference of two timestamps taken in two processes, so it admits one check no statistic supplies: it cannot be negative. Applying that check to *every* run rather than only to runs that looked wrong rejected 1,321 of 2,266, including every run behind the result we were about to publish. Nothing in the rejected data looks wrong in aggregate — medians stay positive, intervals stay narrow, effect sizes stay large. The cause is not clock skew: an inversion occurs when a scheduling stall separates an event from the clock read that timestamps it and outlasts the interval being measured, $\Pr[\text{inversion}] = \Pr[\text{stall} > T_{\text{true}}]$. We manipulate both sides. Real-time priority at *fixed* utilisation cuts the rate 7 to 80× across eight matched pairs; lengthening the true transport 77× cuts it further, so a slower path is a *more reliable* measurement. A kernel trace predicts the observed rate to within a third, unfitted. The stall distribution's tail index is near 0.34, so it has no finite mean and instruments built on averages are structurally blind to this failure.

The second is that the measurement can be silently deleted, and this one is not ours. We instrumented the OpenMessaging Benchmark, which computes end-to-end latency from millisecond-grained timestamps and admits a sample only if the difference is positive, counting nothing that it drops. Across 80 runs we find it computes its reported latency distribution from between 0.36% and 100% of the samples it takes, and reports the same median either way — two distinct values across a 278-fold change in the evidence behind them. Which case obtains is decided by a variable no operator sets: whether the producer's send interval is commensurate with the timestamp resolution. At 500 msg/s — an interval of exactly 2.000 ms, and a rate chosen for being round — every sample in a run shares a phase, and retention across replicates spans 99.5 points. At 457 and 383 msg/s it is stable to 2 points. A three-replicate median, the standard defence, reproduces at one load level in five across three passes of an identical configuration.

What this leaves. We withdraw three claims, including an M/G/1 form our own data refutes and our own earlier reading of these discards as causality violations: in roughly 420,000 discarded samples there is not one negative. Of the original question — does the choice between Apache Kafka and Redis Streams change delay for a live football feed — what survives is modest: the brokers sit within a millisecond, with a reproducible 0.41 ms advantage for Redis that is real and negligible beside a feed whose human annotation latency is measured in seconds. The general consequence is uncomfortable, and it is the same for both failure modes: they bind hardest exactly where the measured effect is smallest, which is to say on the delicate comparisons such papers exist to make.

CCS Concepts: • **Computer systems organization → Distributed architectures**; • **Software and its engineering → Software performance**.

Additional Key Words and Phrases: streaming systems, latency benchmarking, measurement validity, timestamp resolution, quantisation, sampling artefacts, causality violation, Apache Kafka, Redis Streams, reproducibility

1 Introduction

Message brokers carry event data between producers and consumers in most real-time analytics pipelines, and two products dominate the role. Apache Kafka [14] stores messages in a partitioned,

Author's Contact Information: Gustavo Pedro Ricou, School of Computer Science and Statistics, Trinity College Dublin, Dublin, Ireland, pedrorig@tcd.ie.

replicated log written to disk, designed for durability and high throughput. Redis Streams [26] keeps them in memory in an append-only structure, designed for minimum delay per operation. Practitioners choose between them largely on grey-literature comparisons that report Redis as several times faster [6, 12, 38], none of which is reproducible and none of which uses a workload from the domain the reader works in.

1.1 The original question

This work began as a straightforward attempt to fix that, for one domain:

> *Using the publicly available StatsBomb open dataset (2003–2023) and open-source load-generation scripts, compare end-to-end lag between Redis Streams and Apache Kafka for real-time sports data feeds, under varying concurrency.*

The domain supplied something a synthetic benchmark cannot: a real arrival process, and a real answer to *how many feeds run at once*. Football match records carry kick-off times, so the number of simultaneous feeds is an empirical quantity rather than a swept parameter. We treat that workload throughout as the *setting* that produced our findings rather than as a contribution in itself, and Section 3 describes it only to the depth needed to interpret the results.

1.2 The answer we obtained first

Our first campaign returned a clean result. Replaying distinct real matches across one, five and ten concurrent feeds, Redis broker delay rose monotonically (1.006 → 1.197 → 1.346 ms) while Kafka stayed flat to within 0.3%. Every comparison was significant after Holm correction [10], the strongest at $p = 9.0 \times 10^{-11}$, and at five feeds and above the two systems' run distributions did not overlap at all.

The finding was not merely significant; it was explicable. A Redis server executes commands on a single thread, so concurrent streams contend for one execution context, whereas Kafka's partitioning admits genuine parallelism. The measurement appeared to reproduce the textbook architectural distinction between the two designs, in the predicted direction and at the predicted scale.

It was an artefact. Broker delay is computed as consumer receipt minus broker acknowledgement, two timestamps taken in two processes. A message cannot arrive before it is sent, so a negative value is not measurement noise but proof that the instrument has failed. Checking every run against that constraint — rather than only the runs whose results looked implausible — rejected 862 of 1,382 single-machine runs and 459 of 884 multi-machine runs, including every run behind the result above.

1.3 A second failure, in software we did not write

A failure only we have made is a cautionary tale rather than a finding, so we instrumented the OpenMessaging Benchmark [24], the most widely used cross-broker benchmark, and ran it against our own broker. It computes end-to-end latency from Kafka's `CreateTime` timestamps, which are millisecond-grained, and admits a sample to its histogram only if the difference is positive. Nothing counts what fails that test.

On a path whose true latency is a fraction of a millisecond, most differences are exactly zero. Across 80 instrumented runs the benchmark computed its reported latency distribution from between 0.36% and 100% of the samples it took, and reported a median of 1.0 or 2.0 ms either way: two distinct values across a 278-fold change in how much evidence stood behind them.

The variable that decides which case obtains is not one an operator sets, or could reasonably suspect. A load generator paces its producer at a fixed interval. When that interval is an exact multiple of the timestamp resolution, every sample in a run occupies the same phase within the millisecond, so either nearly all of them cross a tick boundary before delivery or nearly none do. At 500 msg/s the interval is exactly 2.000 ms and retention across replicates spans 99.5 points; at 457 and 383 msg/s, incommensurate with the tick, it is stable to 2 points. The rate that breaks the measurement is the round number a user would pick.

1.4 Why these failures are worth a paper

Neither failure is visible in the output that a reviewer, or the benchmark's own authors, would examine.

For the inversions, medians stay positive and plausible, confidence intervals stay narrow, effect sizes are large and the direction agrees with theory. Only 5 to 14% of individual events invert — enough to displace a median by a fraction of a millisecond in a measurement whose entire effect is a fraction of a millisecond, and far too little to make any aggregate look wrong. A benchmark can therefore produce a complete, internally consistent and entirely false result set. No amount of statistical care exposes it, because the contamination is systematic rather than random: greater care produces tighter intervals around the wrong number.

For the deletions, the summary is not merely plausible but *stable*: the same median is reported whether the benchmark kept everything or almost nothing, so the one statistic that would reveal the problem is the one statistic that does not move. The usual defence fails too. A three-replicate median — the standard protection against run-to-run noise — reproduced at one load level in five across three passes of an identical configuration, because the quantity being averaged does not have a central value to converge on.

The two share a governing ratio. In both cases a timescale belonging to the instrument is compared against T_{true}, the interval under measurement. For the inversions it is a scheduling stall, and the failure occurs when the stall exceeds T_{true}. For the deletions it is the timestamp's resolution, and the failure occurs when the tick exceeds T_{true}. Both therefore worsen as T_{true} shrinks — which is to say, both bind hardest on exactly the fast paths that broker comparisons are written about, and on exactly the small differences those comparisons exist to resolve. That is the paper's general claim, and we regard it as more consequential than either mechanism alone.

1.5 Contributions

(1) **A physical-consistency audit for cross-process latency benchmarks** (Section 4.5), stated as a rule, applied uniformly to all 2,266 runs, with its threshold sensitivity reported rather than asserted (Section 6). We show the audit binds hardest exactly where the measured effect is smallest, which inverts the usual intuition that large clean effects are the trustworthy ones.

(2) **A demonstration**, which we regard as the strongest of these: a complete, significant, theory-confirming result set that was physically impossible, reported in full so a reader can judge how convincing invalid data looks (Section 5).

(3) **Falsifiable rules, derived and measured** (Sections 4.6 and 7.1). From a model of the failure we derive that inversion probability is $F_\Delta(-T_{\text{true}})$ and test its consequences: it falls as the measured quantity grows, rises with concurrent process count, rises monotonically with utilisation, and — the rule that bears directly on our own first result — an asymmetric instrument leaves a between-system difference that a symmetric one removes. Those hold. We had also claimed the M/G/1 *form* specifically, and we withdraw it: extended to the utilisation range where the candidate forms diverge, M/G/1 fits worse than the mean. The withdrawal is part of the contribution, because what replaces it is not a better curve.

(4) **The mechanism, established by manipulation** (Section 7.3): an inversion occurs when a scheduling stall outlasts the interval being measured, and we manipulate both sides of that inequality. Raising the stamping threads to real-time priority at *fixed* utilisation collapses the rate 7 to 80× across eight matched pairs. Two loads reaching *identical* utilisation — to four decimals, in two campaigns — by different core geometries differ by 2.07× and 2.05×, so utilisation cannot be the variable. Lengthening the true transport 77× *lowers* the rate, the sign no stress-based account predicts. A kernel trace of run-queue delay then predicts the observed rate to within a third, in three arms at two load levels, with nothing fitted between them, and puts the stall distribution's tail index near 0.34 — below one, so it has no finite mean, which is why an instrument built on averages is structurally rather than accidentally blind to this failure.
(5) **A second failure mode, in a benchmark we did not write, with its mechanism established by manipulation** (Section 6.7). A source-level audit of the OpenMessaging Benchmark, at a named commit and with file and line given, finds it computes end-to-end latency as the same cross-process — in a distributed deployment, cross-*host* — timestamp difference, and admits a sample only when that difference is positive, with no counter for what it discards. We made the discard visible and swept it across 80 runs. Three results follow. *(i)* The benchmark computes its reported distribution from between 0.36% and 100% of its samples, reporting a median of 1.0 or 2.0 ms in every case: the headline is insensitive to a 278-fold change in the evidence behind it. *(ii)* Which case obtains is decided by whether the producer's send interval is commensurate with the timestamp resolution. We establish this by manipulation, holding size, load, duration and host fixed: at an interval of exactly 2.000 ms retention spans 99.5 points across replicates, while two incommensurate rates are stable to 2 points and agree with *each other* to under 2 points despite intervals differing by 19%. *(iii)* The standard defence does not work — a three-replicate median reproduces at one load level in five across three passes of an identical configuration.
(6) **A withdrawal of our own strongest external claim** (Section 6.7). An earlier version of this work reported those discards as the same causality violation we report in Section 4.5, on a counter that could not distinguish sign. In roughly 420,000 discarded samples there is not one negative. We report where the refuting evidence was: in our own artefact, from the day we committed it, in a field we had printed and not read. The exposure in the cross-host case remains established by source audit and *not* by observation — across eight attempts at three background-load levels the benchmark's distributed mode never completed a run — and we distinguish the two.
(7) **A correction to our own model** (Section 7.2), pre-registered before the campaign that tested it. The model above treats Δ as one distribution; it is two. A nine-level load sweep falsifies the scale-family form and measures the mixture instead: a narrow jitter core whose width grows 5× from idle to the knee, and a rare descheduling tail whose weight grows 60×. The consequence for a practitioner is sharper than the leading-order model suggests — a benchmark can sit where its measurements look tight while the tail that corrupts them is already growing.
(8) **A second withdrawal, and a test that would have caught it** (Section 7.5): a twentyfold end-to-end gap we had reported, and built a recommendation on, turned out to be a per-run start-up cost read as a per-event constant, because the runs behind it matched a median of seven events. We give a controlled discriminator — sweep the observation window and count the affected events rather than averaging them, since a per-run cost holds that count fixed while a per-event one grows it — and trace the cost to a single blocking first `produce()` and the four events that queue behind it. The integrity check does *not* catch this, which is the

point: causal consistency is necessary and not sufficient, and a percentile over single-digit samples describes the harness rather than the system.

(9) **A configuration result** (Section 7.7): each system has exactly one client-side setting worth one to two orders of magnitude in delay, and both are free on a co-located testbed.

2 Background and Related Work

2.1 Streaming benchmarks

Stream processing has a mature benchmark literature. Linear Road [3] is the canonical application benchmark; NEXMark [36] models an online auction and has become the standard query suite for dataflow engines; ESPBench [8] constructs an enterprise workload; RIoTBench [30] supplies IoT dataflows; and the OpenMessaging Benchmark [24] measures broker throughput and delay directly across messaging systems. Stonebraker et al. [32] set the requirements these evaluate against.

Two properties of this body of work matter here. Each drives its system with a synthetic arrival process chosen for analytical tractability. And none, to our knowledge, reports what fraction of its runs it discarded on integrity grounds.

The specific comparison we began with is, as far as we can establish, unpublished in the peer-reviewed literature. Practitioner comparisons of Kafka against Redis Streams [6, 12, 38] are numerous and disagree with one another — some report Redis several times faster on throughput, others report Kafka better in the tail — and none states a replay rate, a rejection rule, or an equivalence margin. That absence is what made the original question look worth asking, and it is also why Section 7.4 reports both halves of our own answer rather than the flattering half.

There is precedent for the kind of result we report. Schroeder et al. [27] showed that whether a load generator is open or closed — a modelling choice most benchmarks make implicitly — changes measured behaviour so much that results obtained under one model do not transfer to the other. Their contribution was not a new system but a demonstration that a widespread methodological choice silently determined published conclusions. Ours is of that shape: an unexamined property of how benchmarks record time, shown to determine the result. Mohammad [23] surveys benchmark practice in Kafka event-streaming systems and finds reported results frequently not comparable, because client configuration, acknowledgement semantics and instrumentation differ between the systems under test. Our experience supports that critique and extends it: comparable configuration is not sufficient, because the instrument itself can fail silently under load.

2.2 Measuring time across processes

Lamport [17] established that a distributed system has no single physical time, only a partial order induced by causality. Our check is the simplest application of that principle: a broker's acknowledgement causally precedes the consumer's receipt, so a measurement in which receipt is timestamped earlier violates the causal order and supports no conclusion.

The engineering problem is well known and partially solved. Systems approximate physical time with synchronisation protocols, of which NTP [22] is the most widely deployed; Spanner [5] goes further and exposes clock *uncertainty* to the application, on the grounds that a system which pretends to know the time exactly will eventually be wrong undetectably. Closest to our setting, Cloudprofiler [37] addresses precisely the inadequacy we encountered — that millisecond-grade synchronisation is too coarse to profile modern streaming engines — by calibrating time-stamp counters across nodes during quiescent periods, reaching roughly 92 μs accuracy.

Concurrent work reaches the same observable by a different route. While this study was in progress, Sharma et al. [29] reported the same failure in distributed AI inference pipelines, and their terminology converges with ours: they count *negative timing spans*, defined as timestamps implying that events were received before they were sent. They inject clock skew at the application level across a Kafka and a ZeroMQ pipeline and find a threshold — no violations at 0, 1, 2 or 3 ms of skew, clear violations from 5 ms upward. They also propose a binary `causality_health` signal that declares loss of causal trust when any violation occurs in a rolling window, which is independently the same instinct as the gate in Section 4.5. Two groups arriving at the same check from different directions is, we think, the strongest available argument that the practice is missing rather than merely unfashionable.

Our result qualifies theirs in one respect that matters for practice. Their baseline establishes that with clocks properly aligned, *no negative spans occur* — the finding that licenses reading their onset region as a safety threshold. We observe inversion rates near 23% with no clock skew available to blame: our transport measurement takes both stamps on a single host, and the inter-host offset on our testbed measures 0.067 ms, twenty times below their lowest tested skew and fifty times below their onset. The difference is load. Their pipeline runs on hosts with "sufficient memory to support inference workloads without swapping or resource contention"; ours sit at 88% utilisation, where Section 7.3 shows the scheduler supplies stalls far longer than the interval being measured. A deployment that keeps skew under 3 ms and concludes it is safe would be wrong on a loaded machine, and the mechanism that makes it wrong needs no skew at all.

We therefore make a narrower claim than "nobody has considered this". The accuracy problem is recognised and better instruments exist. What appears to be missing is the *practice*: an audit applied uniformly to every run of a campaign, with the rejection rate reported the way a survey reports its response rate. Cloudprofiler improves the instrument; we argue that whatever instrument is used, its output should be checked against causality and the check should be published. Our own failure is also not one of clock synchronisation — both processes read the same operating-system clock on one machine — but of *when the timestamp is taken*: a thread descheduled between the event and the call recording it stamps a time later than the event, and under CPU saturation that delay routinely exceeds the interval being measured.

Two 2026 results sit close to ours and we are explicit about what each already establishes. Chandrasekar and Kramberger [4] identify a systemic client-side measurement bias in production LLM inference benchmarks — a single-process client that cannot keep up under concurrency and inflates the latencies it reports — and model it as an M/G/1 queue, a framing we adopt in Section 4.6. Sharma et al. [29] go further in our direction: they report *negative timing spans* in a multi-node inference pipeline, including one built on Kafka, and show that causality violations emerge from a few milliseconds of clock skew while the system continues to function correctly.

We therefore do not claim that causality-violating measurements are a new observation. Three things remain open after that work, and they are what this paper supplies. First, Sharma et al. *inject* clock skew between nodes; our violations arise with no skew at all, on a single machine where both processes read the same clock, purely from scheduler delay in reaching the clock. That is a harder failure to defend against, because synchronising clocks does not remove it. Second, neither paper states a detection rule, applies it uniformly, or reports what fraction of a campaign it rejects. Third, and most importantly, neither relates violation probability to the magnitude of the quantity being measured. Section 4.6 derives that relation and Section 6.6 shows what follows from it: the check is least forgiving exactly where a benchmark is most delicate.

One further difference decides how each failure can be caught at all. A load-induced inflation is wrong but physically possible, so detecting it needs a better instrument or an independent reference. A causality violation is self-detecting, at no cost and with no reference. That property, rather than the existence of load-induced bias, is what we build on.

A third 2026 result approaches the same problem from the opposite direction and supports the premise we build on. Swami and Chougule [33] show, for edge inference systems, that deployment interference corrupts not only the timing being measured but the *timing observability infrastructure that measures it*, and that the two failures occur independently. They propose no detection rule and report no rejection rate, so the practice gap above is unaffected; what they establish is that a measurement instrument failing under exactly the load it is deployed to measure is not peculiar to our harness or our domain.

Jain [11] sets out the standard requirement that an instrument be validated on the system it will measure. The check in Section 4.5 is a cheap, specific instance of that for cross-process delay.

2.3 Resolution, discarded samples, and sampling artefacts

The second failure mode we report (Section 6.7) is not about *when* a timestamp is taken but about *how finely* it can express the answer, and about what a benchmark does with the samples that fall below that granularity. Three literatures sit close to it. None reports it, and we are specific about the relation to each.

Coordinated omission is the closest neighbour, and is the mirror image. Tene's *coordinated omission* [35] is the canonical result that a load generator can be systematically optimistic: a synchronous measurement thread blocks while the system under test stalls, so the samples that would have been worst are never taken at all, and reported percentiles are wrong by orders of magnitude. The failure we report inverts every clause of that description. The samples *are* taken and the latency *is* computed; what removes them is a guard downstream that rejects the result. Coordinated omission loses the slow tail; this loses the fast bulk. Both end with a confident summary computed over a biased subset, which is why they belong together — and why treating them as the same thing would be wrong, since a correction for one does nothing about the other.

That last point is not hypothetical. The benchmark we instrument records into an `HdrHistogram` [34], which ships explicit machinery for coordinated omission in the form of `recordValueWithExpectedInterval`. The ecosystem therefore already carries a correction for the known sampling bias and none for this one. There is a further irony worth stating plainly: `HdrHistogram` rejects negative values, which is a sound defence against the failure of Section 4.5, and it is precisely that defence which motivates the positivity guard whose `else` branch silently deletes the sub-tick samples. A protection against one measurement artefact created the conditions for another.

Dithering: the remedy is old, and belongs to a different field. Breaking periodic sampling artefacts by randomising or offsetting the sampling instant is standard practice in signal processing, where dithered measurement rates are used precisely to separate genuine components from aliasing products. Our incommensurate-rate arms are dithering, arrived at as a diagnostic rather than borrowed as a technique. The recommendation we make to benchmark authors in Section 8.1 is therefore not new technology but an old one never applied here: do not pace a load generator at a rate commensurate with the resolution of the clock you are subtracting, and if the rate is fixed by the experiment, dither it.

Granularity is discussed as staleness, not as deletion. The relationship between a fixed send interval and a coarser clock tick is recognised in the timing literature, but as a bound on how *stale* a reading may be. What is missing is the consequence when a positivity guard sits downstream of the subtraction: staleness becomes deletion, and because the phase is common to every sample in a run, the deletion is all-or-nothing across the whole run rather than a uniform thinning of it.

What we claim as new, stated narrowly. Not that timestamp quantisation is unknown; it is elementary, and Cloudprofiler [37] exists because millisecond granularity is understood to be too coarse for this class of system. The contribution is the *interaction*: a quantised timestamp, plus a producer paced at an interval commensurate with that quantum, plus a guard that admits only positive differences, together produce a retention fraction that is bimodal, irreproducible between runs of an identical configuration, and invisible in the reported summary. We have found no report of that combination, and it occurs at 500 msg/s, a rate whose only distinction is that it is round.

2.4 Where scheduling delay comes from, and why it is not one distribution

Our model treats the stamping delay δ as scheduler waiting time, and Section 7.2 finds it has two components rather than one. Both the framing and the correction are anticipated by the tail-latency literature.

Why the thread waits while cores look free. Section 7.3 finds that spread load starves the stamping thread more than concentrated load at identical utilisation, which presumes that a runnable thread does not simply migrate to whichever core is idle. That presumption is not ours: Lozi et al. [20] document four distinct Linux scheduler bugs under which cores stay idle *for seconds* while runnable threads sit in another core's run queue, and note that conventional testing was ineffective at finding them because throughput looks healthy throughout. Our measurement is a consequence of that behaviour observed on the instrument rather than on the workload: if work conservation held exactly, a stamping thread would never wait while a core was free, and the geometry contrast of Table 14 would have to be flat. It is not flat, and it converges only where no core is free to migrate to.

That literature also supplies the shape. Section 7.3 measures a tail index near 0.34, meaning the stall distribution has neither a finite mean nor a finite variance, which is why the cumulative scheduler counters could not account for the effect — they estimate a quantity that does not exist. Interference-driven tails of this severity are consistent with what the tail-latency literature reports, though we know of no prior measurement of the index itself for run-queue delay, and we report ours as one fitted value from one campaign rather than as a constant of the system.

Li et al. [18] decompose the tail latency of three Linux servers into hardware, operating-system and application-level sources, and report the finding that matters most for us: measured tails are *substantially worse* than a queueing model alone predicts, with the excess traced to interference from background processes, request re-ordering under constrained concurrency, interrupt routing, power management and NUMA effects. A single-parameter M/G/1 description is therefore a leading-order account, not a complete one — which is precisely what Section 7.2 measures directly on the instrument rather than on the system under test. Their background-interference mechanism also predicts the temporal signature we observe: a core blocked by another task delays not one event but every event arriving in that interval, so the resulting failures should appear in consecutive bursts. Section 8.4 tests that prediction and finds exactly this clustering.

Our contribution here is therefore not the observation that scheduling delay is heavy-tailed, which is established, but that this shape propagates into the *measurement* of an unrelated quantity, and that load moves the tail's weight far faster than the core's width (Table 12). The practical consequence is specific: a benchmark can sit at a utilisation where its measurements look tight — the core is narrow — while the tail that actually corrupts them is already growing.

2.5 Statistical methods

Latency distributions are non-normal, so difference tests are rank-based: Mann–Whitney U [21] per condition and Kruskal–Wallis [15] across concurrency levels, with Holm's correction [10]. Because several claims here are negative, we state them with two one-sided tests [16, 28] against a pre-specified margin rather than by failing to reject equality. As a point estimate of the difference between two systems we use the Hodges–Lehmann shift [9] with percentile bootstrap intervals [7];

Section 6.4 shows that this estimator's breakdown point is what makes our main conclusion robust to the audit's own side effects.

3 The Setting

The workload is a live football event feed, and we describe it only as far as the results require. Full characterisation of 3,315 matches from the StatsBomb open dataset [31], spanning 52 competition-seasons from 2003 to 2023, is available in the artefact; event data of this kind is collected by trained human operators, whose accuracy has been studied [19, 25].

Three properties of the workload drive every experimental choice (Table 1). We name, for each, the later result that depends on it, because a setting section that cannot say what it is for should not be in the paper. A reader who wants only the measurement contribution can note that the workload supplies three things a synthetic generator would not have supplied — a sparse arrival process, a burst at a known instant, and an empirically bounded concurrency — and skip to Section 4.

It is sparse. A match produces a median of 3,507 events over roughly 8,412 seconds, a mean rate of **0.415** events per second. This is four orders of magnitude below the rate at which either broker begins to strain, and it is the property that makes Section 7.5's result possible.

It is bursty, and one burst has a known instant. Over a sliding ten-second window the same feeds peak at 2.70 events per second, a peak-to-mean ratio of 6.45; half of all inter-arrival gaps are one second or shorter. The feed is idle most of the time and busy in short bursts. One of those bursts is special: a kickoff is recorded as a pass, a ball receipt and a carry inside the same second, so *every* match in our corpus opens with at least five events sharing t_{sim}=0. That coincidence — the densest burst arriving when the producer is coldest — is what turns a one-off client start-up cost into the median of a short run, and it is the mechanism behind the withdrawal in Section 7.5. A synthetic Poisson generator has no kickoff, and would not have produced it.

Its concurrency is bounded and knowable. Match records carry kick-off times, so the number of simultaneous feeds is empirical rather than swept. Across 2,346 kick-off instants the largest carries **12** simultaneous matches, and at most **21** are in play at once. Domestic leagues schedule the final matchday simultaneously by rule, so a 20-team league plays ten concurrent matches. We therefore benchmark at $N \in \{1, 9, 10, 12\}$: the median slot, the upper quartile, the structural league bound, and the observed maximum. This is what keeps the concurrency axis of Section 7.4 from being an arbitrary sweep — the levels are measured facts about the domain, not round numbers. Two limits apply. The dataset is a sample, so observed simultaneity is a lower bound; and the bound is per league, so a platform serving many competitions faces their sum.

Taken together these also explain why the broker question turned out to be the least interesting thing we could have asked, and we would rather say so here than let a reader discover it in Section 7.4. At 0.415 events per second and at most a dozen concurrent feeds, this workload sits four orders of magnitude below where either broker strains. The domain that made the failure visible is the same domain in which the original question has a boring answer.

4 Method

4.1 Testbeds

Because several of our earlier numbers proved to be properties of a machine rather than of either broker, we state both testbeds and measure the floors that bound resolution.

Testbed A (single machine). Windows 11, 8-core AMD Ryzen, 31 GB RAM, CPython 3.9. Producer and consumer are native processes; Kafka 4.1.1 and Redis 7.2.4 run in Docker containers whose engine executes in a WSL2 virtual machine, reached through published ports. Two floors bound it:

Table 1. The workload, from 3,315 matches. Medians across matches; peak rate is the maximum over any sliding ten-second window.

Property	Value
Events per match	3,507
Match duration	8,412 s
Mean arrival rate	0.415 ev/s
Peak rate (10 s window)	2.70 ev/s
Peak-to-mean ratio	6.45
Median inter-arrival gap	1.0 s
Distinct kick-off instants	2,346
Max simultaneous kick-offs	12
Max matches in play at once	21

a requested 1 ms sleep takes a median of 15.6 ms (the Windows timer tick), and opening a TCP connection to a published port costs 5.6–9.9 ms.

Testbed B (four machines). Four Oracle Cloud `VM.Standard.E5.Flex` instances — three brokers and one driver — on Ubuntu 22.04, CPython 3.10, over a private network, with client libraries pinned identically to Testbed A. Both floors disappear: a 1 ms sleep takes 1.06 ms, and a broker round trip on an established connection is 0.22 ms locally and 0.24 ms across machines. A genuine two-machine network is thus *faster* than Testbed A's co-located path, which is the sharpest statement we can offer of how much a testbed distorts a benchmark. Every result reported as a finding comes from Testbed B.

4.2 Measurements

Each match is preprocessed into a replay plan recording, per event, the time at which it should be emitted. A producer replays a plan at a configurable speed-up; a consumer records arrivals. In the concurrency experiment each of the N feeds carries a *distinct* real match at its true rate: an earlier design gave every feed the same merged ten-match plan, which made the concurrency axis covertly a throughput axis. The plan corpus holds eleven matches, taken from one real slot of simultaneous kickoffs, so the mapping is one-to-one at every level we report except N=12, where the twelfth feed reuses the first plan. We state this because "distinct" is otherwise exact and the one exception is load-bearing for nothing: the repeated plan is replayed by a separate producer against the same broker, so it adds a feed's worth of connections and events either way.

The primary measure is **Time-to-Insight** (TTI), from an event's scheduled emission to its availability at the consumer. We decompose it (Figure 2):

$$\mathrm{TTI} = \underbrace{(t_{\mathrm{send}} - t_{\mathrm{sched}})}_{\text{scheduling lag}} + \underbrace{(t_{\mathrm{recv}} - t_{\mathrm{ack}})}_{\text{broker transport}} + \mathrm{residual}.$$

Scheduling lag is measured entirely inside the producer and characterises the client. Transport spans the broker and characterises the product. The distinction is not cosmetic: for this workload the two have opposite answers, and reporting only the aggregate is what concealed the result in Section 7.5.

Critically, t_{ack} is stamped in the *producer* process (from Kafka's send callback, or on return from Redis's `XADD`) while t_{recv} is stamped in the *consumer*. That subtraction is the one that can go wrong.

Campaign	Manipulated	Held fixed	What it settles
E1 concurrency	feeds N: 1, 9, 10, 12	rate, host, plan set	the original question: does broker choice matter?
E-B delay sweep	injected delay 0-50 ms	load, feeds	H1 effect size (direction only; confounded)
E-A / E-A3 load sweep	background load 0-12	rate, feeds, plan	H2 utilisation, H10 mixture, F_Δ recovery
E-A2 process count	processes at fixed rate	aggregate event rate	H4 oversubscription
E-C3 / E-C4 stamping	ack stamp: callback vs inline	load, in-flight, backend	H3 asymmetry (replicated)
window sweep	observation window 60-600 s	rate, host, match	start-up cost vs per-event constant
transport (x2)	feeds, powered	verified real-time rate	broker transport, equivalence + shift
E-A5/A5b/A7 stamping priority	SCHED_FIFO on the stamping threads	utilisation, to 0.003	scheduling, not utilisation (8 pairs, 7-80x)
E-A6 / E-A6b load geometry	cores free vs all duty-cycled	achieved utilisation	utilisation is not the variable (2.07x, 2.05x, at rho 0.7531)
E-A9 run-queue trace	nothing: observation only	load, priority arm	P(stall > T_{true}) predicts the rate, unfitted
E-A10 transport sweep	payload size; T_{true} 77x	load, hosts, code path	other side of the inequality; tail index 0.34
E-A8 co-location	broker on the driver	utilisation, to 0.002	nothing: transport did not move, so it is withheld
OMB external	instrumented discard counter	our broker, under load	the exposure is not ours alone

Every claim in Section 7 traces to one row. No row both generates and tests the same hypothesis, and a row that settled nothing says so.

Fig. 1. The experiment map. Each campaign manipulates one variable, holds the others fixed, and settles one claim. The delay sweep is marked confounded because injecting delay also inflates variance (Section 8.4), so we use it for direction only.

4.3 The experiments, and what each one settles

Our campaigns accumulated over three months, each added in response to a defect or a question the previous one raised, so their names do not reveal their logic. Figure 1 states it in one place: what each campaign manipulates, what it holds fixed, and which claim it is the evidence for. Two properties of that map are deliberate. Every claim in Section 7 traces to exactly one row, so a reader can check that nothing is asserted without an experiment behind it. And no row both generates and tests the same hypothesis: where a pilot analysis suggested a hypothesis, a fresh campaign tests it, and Section 7.2 says so explicitly.

4.4 Configuration, and why each setting is what it is

Mohammad [23] finds that published Kafka comparisons are frequently not comparable because client configuration and acknowledgement semantics differ silently between the systems being

Table 2. Every configuration that can move a latency number, and why it holds that value. Settings marked *learned* were discovered by a defect they caused, not chosen in advance.

Setting	Value	Why
Kafka producer		
`acks`	`all`	Durability semantics matched to Redis's synchronous `XADD`; weaker acknowledgement would compare different guarantees.
`linger.ms`	0	No artificial batching delay; the workload is sparse, so any linger would dominate the quantity measured.
`max-inflight`	64	*Learned.* At the default 1 the client blocks on a round trip per event, making the comparison one of harnesses rather than brokers (Section 7.7).
`compression`	none	Football events are small JSON; compression would add CPU to the path being timed for no representative benefit.
`ack-stamp`	`callback`	Default, and deliberately asymmetric with Redis; Section 7.1 measures what the asymmetry costs by also running `inline`.
Redis producer		
send workers	pool	Non-blocking dispatch so the emission loop keeps schedule; send and acknowledgement are stamped in the worker, preserving a valid transport span.
Consumers		
Redis `ack-batch`	200	*Learned.* Acknowledging per message adds one network round trip per message, which at 20 ms of delay is the difference between 103 ms and 4,138 ms (Section 7.7).
Redis `count`	200	Read batch; matched to the acknowledgement batch so neither dominates.
Kafka `poll-timeout`	1000 ms	Client already amortises fetches; no per-record round trip exists to remove.
Replay		
speed-up	derived	*Learned.* Plans carry a baked-in 120× compression, so the flag is not the rate; it is derived from the plan and verified against wall time before every campaign (Section 6.5).
window	60–600 s	Swept deliberately: the number of events a run contains is itself a variable, and Section 7.5 is the reason.

compared. We therefore state every setting that can move a latency number, with the reason it holds the value it does (Table 2). Two of these were not chosen but *learned*: the pipelining setting and the acknowledgement-batching setting each cost us a result before we understood them, and both are the subject of Section 7.7.

The general principle is that the two clients must be equalised on *semantics*, not on parameter names. Kafka and Redis expose different knobs, and matching them syntactically is what produces the asymmetries this paper is about. Concretely, Kafka's producer buffers and pipelines by default while our Redis producer dispatches to a worker pool; leaving `max-inflight` at 1 makes the Kafka client wait for a broker round trip per event while the Redis client does not, which is not a broker difference but a harness difference.

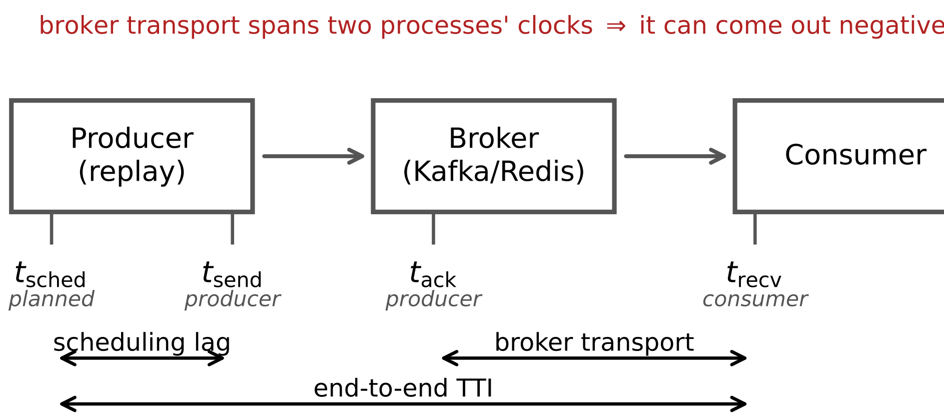

Fig. 2. The four timestamps and the intervals each measure spans. Scheduling lag is measured within one process. Broker transport subtracts a producer-side timestamp from a consumer-side one, and can therefore come out negative when either process is delayed.

4.5 The consistency check

We implement the causal constraint as a stated rule, fixed before the final campaign and applied to every run including those whose results looked reasonable. A run is **rejected** if either

(1) more than 1% of its events carry a negative value in any latency component, or
(2) the median of any component is negative.

A condition is usable only if all of its runs survive. Where a condition is partly rejected we report the retention rate, and Section 6.4 bounds the selection this introduces. The tool exits non-zero when any run fails, so a campaign script can stop on it.

4.6 A model of the failure, and what it predicts

A rule that rejects runs is only useful if one can say *when* it will fire. This subsection states the model that makes that prediction, because it is what converts the check from a hygiene step into something a reader can apply to their own measurement without repeating our experiment.

Every timestamp is taken by software some interval after the physical event it claims to mark, because the thread that reads the clock must first be scheduled. Writing those stamping delays as δ_{ack} and δ_{recv},

$$T_{\text{measured}} = T_{\text{true}} + \Delta, \qquad \Delta \equiv \delta_{\text{recv}} - \delta_{\text{ack}}, \tag{1}$$

so an inversion occurs exactly when $\Delta < -T_{\text{true}}$, and

$$\Pr[\text{inversion}] = F_{\Delta}(-T_{\text{true}}). \tag{2}$$

Figure 3 draws both halves of Equations 1 and 2. Panel (a) shows the mechanism: the acknowledgement is stamped late enough that the consumer's receipt timestamp precedes it, even though the message plainly arrived after it was sent. Panel (b) shows the consequence that matters for benchmark design.

Equation 2 says something that is easy to miss and, we think, the most useful thing in this paper. **Inversion probability is a property of the quantity being measured, not only of**

the machine. The same instrument, on the same host, under the same load, is more likely to be invalid when measuring a small latency than a large one, because a fixed distribution of Δ overlaps a small T_{true} almost entirely and a large one hardly at all. This is why our network-delay arm, where transport is tens of seconds, passes the check on every run while same-hardware arms measuring one millisecond fail wholesale.

Three further predictions follow — on utilisation, on process count, and on instrument asymmetry — and Section 7.1 reports the experiments that test them.

Utilisation, and what drives it. δ is the waiting time of a runnable thread. Treating the CPU as a server, the Pollaczek–Khinchine result for an M/G/1 queue gives mean waiting time growing as $\rho/(1-\rho)$ in utilisation ρ. The spread of Δ should therefore be small and flat at low utilisation and grow sharply near saturation: a *knee*, not a linear ramp. This is the same framing Chandrasekar and Kramberger [4] apply to client-side bias in inference benchmarks. A corollary sharpens it into a second prediction: ρ is set by the number of *runnable threads* per core, not by the offered event rate, so at a *fixed* aggregate rate, adding concurrent producer/consumer processes should raise inversion rate — the failure follows process count, not load.

Asymmetry. It is $\mathrm{E}[\Delta]$, not its variance, that biases a *comparison*. Symmetric instrumentation gives $\mathrm{E}[\Delta] \approx 0$: noise, no systematic error. Asymmetric instrumentation does not — and if two systems under comparison stamp differently, the instrument manufactures a difference between them. This is precisely our situation: Kafka's producer stamps the acknowledgement in an asynchronous callback, Redis's on return from a blocking command. It is the most plausible account of how our first result set acquired a large, significant, entirely spurious effect.

4.7 What the check cannot catch

A paper about measurement validity should be explicit about the limits of the instrument it proposes, so we state them before reporting any result with it.

Equation 2 makes the blind spot precise. The check fires only when $\Delta < -T_{\text{true}}$: it detects the tail of the asymmetry distribution that crosses zero, and nothing else. Three consequences follow, and the third is the serious one.

It cannot see bias smaller than the measurement. If Δ displaces every measurement by, say, 0.3 ms while T_{true} is 1 ms, no value goes negative and the check passes a corpus whose every number is 30% wrong. Passing is therefore evidence that the instrument did not fail *catastrophically*, not evidence that it was accurate. We use the word "sound" in this narrow sense throughout.

It says nothing about the tail it does not reach. A run just above the threshold and a run just below it may differ arbitrarily in bias. The rejection rate is a hygiene statistic, not an error bar.

It is blind to symmetric inflation. The failure Chandrasekar and Kramberger [4] describe — a saturated client inflating every latency it reports — keeps all measurements positive and passes our check completely. Their failure mode and ours are complementary, and neither instrument detects the other. A benchmark that applies only the check proposed here is still exposed to theirs.

This is why we frame the contribution as a *necessary* condition rather than a validation procedure. A corpus that fails the check is unusable. A corpus that passes it has cleared the cheapest available bar and nothing more.

4.8 Instrumenting a benchmark we did not write

The second failure mode is measured in the OpenMessaging Benchmark rather than in our harness, so the instrument is a patch to somebody else's code and its design needs stating.

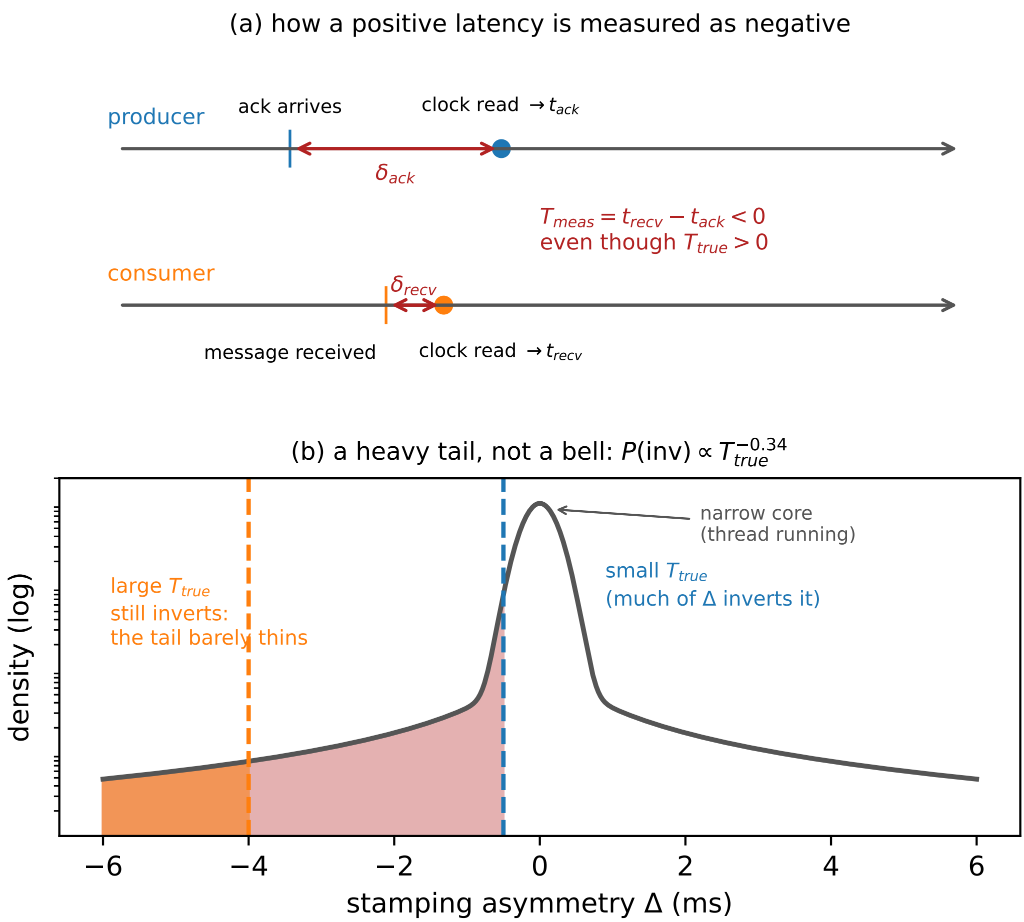

Fig. 3. (a) How a positive latency is measured as negative: each side stamps its clock some interval after the event, and if the producer's delay exceeds the true transport plus the consumer's delay, the difference inverts. (b) Why the check bites hardest on small effects. The same distribution of stamping asymmetry Δ inverts most of a small T_{true} and almost none of a large one.

What the patch does, and what it deliberately does not. At the named commit, a sample is admitted to the histogram only by `if (endToEndLatencyMicros > 0)`. We add counters in the existing `else` branch and change nothing else: not the latency computation, not the guard's condition, not any reported statistic. The benchmark's own output is therefore exactly what it would have been unpatched, which is the property that lets us set its published summary beside the retention behind it. The diff is in the artefact.

The counters must distinguish sign, and an earlier version did not. Both a causality violation and a sub-millisecond delivery fail a `> 0` test. A single unsigned total cannot tell them apart, and we initially reported one, drawing a conclusion the sign-separated data later refuted (Section 6.7.1). The counters are now four — zero, negative, most-negative, kept — and the distinction between them carries the finding.

Counts must be exact, which required two corrections. The counters are declared `static`: an embedded run constructs several `WorkerStats` instances, and per-instance counters silently split the totals between them. Totals are printed once by a JVM shutdown hook rather than read from periodic progress lines, because those fire every 10,000 discards and reading the last one quantises the answer to a multiple of 10,000 — three runs with genuinely different totals all reported 50,000 that way. Every cell records which source its counts came from, and cells whose counts are quantised are excluded from every analysis by rule rather than by inspection.

A validity gate that refuses to report a number. A benchmark that fails to run discards nothing, and a zero from such a run looks exactly like a measured zero. Our first distributed attempt wrote `discarded=0` after the worker died on a classpath fault, and that vacuous zero reached a draft of this paper as a genuine negative result. The campaign now validates before writing: the run must have produced publish-rate output and no failure lines, and if it did not, the row is written `valid=0` with the reason and no count. The gate has since refused eight distributed attempts (Section 6.7.5), and its refusals are in the artefact alongside its acceptances.

Denominators. OMB warms up for one minute by default, resets its histograms at that boundary, and reports percentiles over the test phase alone; our counters are never reset and would otherwise span both. We make the setting explicit and report a matched sweep with warmup disabled. It changes the sample count per cell from $\approx$ 120,000 to $\approx$ 90,000 and leaves the retention fraction unchanged, so the figures computed with warmup included stand.

Campaign design. Retention at a fixed configuration proved not to have a stable central value (Section 6.7.4), which dictates the design: every claim is made over whole distributions of replicates rather than over a summary of them, and the axes are swept one at a time with everything else held fixed — background load, message size, producer rate, and the warmup setting — because the variable that turned out to matter was one we did not initially think to hold fixed.

Every run is indexed, including the failures. Each cell is one row in a ledger built by reading the cell's own log rather than a receipt written afterwards, recording configuration, counts, whether the counts are exact, and the size and SHA-256 of the log they came from. This is not bookkeeping for its own sake: it recovered a valid cell whose script died between finishing the benchmark and writing its result row — a cell that is the median of its load level — and it exposed a fabricated negative sample that our own indexer had produced by mapping an older result schema positionally. A campaign that silently drops its failures is a selection of the runs that worked, which is the failure this paper is about, one level up.

4.9 Statistics

Margins are fixed before testing and proportionate to the quantity compared: $\delta = 1$ ms for broker transport, a quantity of order one millisecond, and $\delta = 40$ ms for end-to-end TTI, one video frame at 25 fps. Holm families are declared over the design actually executed: the four per-N transport comparisons form one family, the network-delay arm another; omnibus Kruskal–Wallis tests are uncorrected; equivalence tests are pre-specified and belong to no family. Surviving samples range from 8 to 35 runs per cell; the 8-run cell is treated as descriptive and flagged wherever it appears.

Because sample size differs sharply between the historical corpus and the campaigns that came after it, we state which claim rests on which rather than leaving a reader to infer it (Table 3). The distinction matters for one argument in particular: the retention-bound defence of Section 6.4 holds only while more than half a cell survives, and its tightest cell clears that line by 2.8 points. That argument is needed for the E1 corpus alone. Every campaign run after it retains all of its runs, so no later claim depends on the bound.

Table 3. What each claim rests on. E1 is the only corpus where retention is a live concern; the campaigns after it retain every run, so the breakdown-point argument of Section 6.4 is needed for one row only.

Claim (section)	Campaign	Runs per cell
Audit rejection rates (6)	all corpora	2,266 runs total
H1 effect size (7.1)	co-located vs network	15 per delay level
H2 utilisation shape (7.1)	E-A, E-A3	25 per load level
H4 process count (7.1)	E-A2	15 per level
H3 asymmetry (7.1)	E-C3, replicated E-C4	10, then 15 per arm
Mixture structure (7.2)	E-A3, nine levels	25 per level
Scheduling, not utilisation (7.3)	E-A5, E-A5b, E-A7	25 per arm
Load geometry (7.3)	E-A6, both geometries	25 per arm
Traced run-queue tail (7.3)	E-A9	25 per arm
T_{true} sweep (7.3)	E-A10, four payloads	25 per level
Broker transport (7.4)	powered, replicated	15, then 8 per cell
Start-up withdrawal (7.5)	window sweep, twice	3 per window
Historical: E1 (7.4)	E1	8–35, *52.8–100% retained*

Table 4. **The first result set, later rejected.** Broker transport (median, ms) by concurrency, Testbed A, 10× replay. Every cell fails the check of Section 4.5.

N	Kafka	Redis	Difference	p_{Holm}
1	0.999	1.006	0.007	1.0×10^{-4}
5	1.001	1.197	0.196	6.5×10^{-6}
10	1.002	1.346	0.344	9.0×10^{-11}

Every inversion rate reported in Section 7 is a proportion over 2,985 matched events per cell — 25 runs of the same replay plan, and identical in every campaign and every arm. That uniformity is worth stating once rather than repeating in each table: it means every rate comparison in this paper, within a campaign or across two of them, is made at equal n, so a ratio between two cells cannot be an artefact of one resting on more data than the other. The per-cell counts are carried in the released artefacts alongside the rates, and the paper's tables are regenerated from those files.

5 The Answer We Obtained First

We present the first result set in full rather than describing it, because a reader cannot otherwise judge how convincing an invalid corpus looks.

5.1 The result

On Testbed A, replaying distinct real matches at ten times real speed across one, five and ten feeds with 15–30 runs per cell and both producers pipelined, broker transport separated the two systems cleanly (Table 4). Every comparison was significant after correction and at $N \geq 5$ the run distributions did not overlap.

5.2 The checks it had already passed

The corpus was not naive. It was the product of five earlier corrections, each of which had reversed the apparent winner and each found by investigating a result that looked wrong:

Table 5. Consistency audit of every run in the study.

Corpus	Runs	Rejected	Conditions	Usable
A (single machine)	1,382	862 (62.4%)	76	8
B (four machines)	884	459 (51.9%)	40	13
Total	2,266	1,321 (58.3%)	116	21

(1) *A process-relative clock.* Cross-process timestamps taken with `perf_counter_ns`, whose origin is per-process, injected each run's consumer start-up offset into every measurement. Fixed with a shared wall-clock epoch.
(2) *A saturating replay speed.* At 120× the producer could not keep pace, inflating scheduling lag to tens of seconds.
(3) *An asymmetric load generator.* The Kafka producer blocked on a broker round trip per event while the Redis producer dispatched asynchronously. Median TTI at one feed fell from 1,644 ms to 16 ms once the Kafka producer was pipelined.
(4) *A concurrency axis that was covertly a throughput axis*, from the merged-plan design described in Section 4.2.
(5) *A heavy tail contaminating every mean.* Kafka's end-to-end 95th percentile sat at 99–105 ms at every concurrency level while its median moved from 1.8 to 7.2 ms. A 95th percentile that ignores load is not system behaviour; decomposition located it in scheduling lag, not transport.

We also verified the comparison was not biased by message size or protocol: payloads are near-identical (median ≈170 bytes) and serialisation costs are negligible and comparable.

A sixth problem shaped our later protocol. Our first run of the acknowledgement experiment of Section 7.6 reported no improvement whatsoever — a clean refutation of our own hypothesis. The treatment had never reached the consumer: the orchestration script accepted the option but, through a failed edit, never passed it on. We found this only by deliberately supplying an invalid option and observing that the consumer did not reject it. A null produced by an unapplied treatment is indistinguishable in the data from a genuine one, so every intervention reported here carries a manipulation check that aborts the campaign if the treatment is not accepted.

The point of the list is not the corrections. It is that a corpus which had survived six of them, a fairness audit, and a full battery of rank tests, effect sizes and multiplicity correction, was still entirely invalid, and nothing in its output said so.

6 The Audit

6.1 What it rejects

Applying the rule to all 2,266 runs rejects 1,321 of them (Table 5). On Testbed A only 8 of 76 conditions survive intact; on Testbed B, 13 of 40. No condition behind Table 4 survives.

6.2 Why it fails, and why it looked correct

The cause is legible in *which* conditions fail: rejection tracks replay acceleration, with the accelerated concurrency conditions failing and the unaccelerated ones passing. We state that as a direction rather than as two rates, because the rate is one thing our own artefacts do not record — see Section 6.5. The mechanism is the one described in Section 2.2. The acknowledgement timestamp is taken by the producer's callback thread; when accelerated replay saturates the driver's CPU that thread is descheduled, and by the time it reads the clock the consumer has already recorded

Table 6. Bounding the unequal-retention selection effect. “Worst case” imputes every rejected Redis run at the largest value observed in that cell. Margin 1 ms.

N	Redis retained	Shift (ms)	Worst case	Equivalent
1	8/8	+0.081	+0.081	yes
9	20/27	+0.021	−0.051	yes
10	17/30	+0.116	−0.485	yes
12	19/36	+0.053	−0.227	yes

receipt. It is a timestamping race under load, not clock skew, which is why medians stay positive at moderate load and invert wholesale only at saturation. At one hundred connections the failure becomes gross enough to see unaided: Kafka’s median transport is reported as −6.4 ms.

The corpus looked healthy for an arithmetic reason. Between 5 and 14% of events invert, which displaces a median by a few tenths of a millisecond in a measurement whose entire effect is 0.34 ms. The bias is also *asymmetric* between arms, because the two clients record the acknowledgement differently — Kafka from an asynchronous callback, Redis on return from a blocking command — and an asymmetric bias between two arms of a comparison manufactures a difference where none exists.

6.3 How much the threshold matters

We expected the distribution of inversion rates to be clearly bimodal, which would have made the 1% threshold uncontroversial. It is not (Figure 4a): only 104 of 1,382 runs are completely clean and the rest spread over three orders of magnitude with no natural gap. The threshold is a real decision, so we report its consequences rather than asserting robustness.

It strongly affects *how many runs* are rejected — from 92.5% at zero tolerance to 19.2% at a permissive 20% (Figure 4b). It does not affect *which conditions are usable*, and therefore changes no conclusion here: a condition requires all its runs to survive, and even at a 5% threshold the accelerated arms retain only 32–77% of theirs. The withdrawal of Table 4 holds across the entire range.

6.4 Bounding the audit’s own selection effect

An audit that rejects unequally between arms introduces selection. In our concurrency experiment Kafka passes on 100% of runs at every level while Redis passes on 53–100%. Since inversion is caused by driver saturation, the surviving Redis runs are plausibly the less-loaded ones, and the bias would run in exactly the direction that manufactures our reported equivalence.

The rejected runs’ values are not recoverable, so a threshold sweep is impossible. Instead we bound: impute every rejected Redis run at a hypothetical value and ask how large it must be before the conclusion changes (Table 6). Equivalence survives at every level, and no imputed value up to 1 second breaks it.

The reason is structural, and so is its limit. The Hodges–Lehmann shift is a median of pairwise differences and has a breakdown point of one half: while more than half the runs survive, nothing the rejected runs could have contained moves the estimate outside the margin. Our tightest cell is N=12 at 52.8% retention — only 2.8 points above breakdown. Had retention fallen below one half this defence would not apply, and we would have had to withdraw the cell.

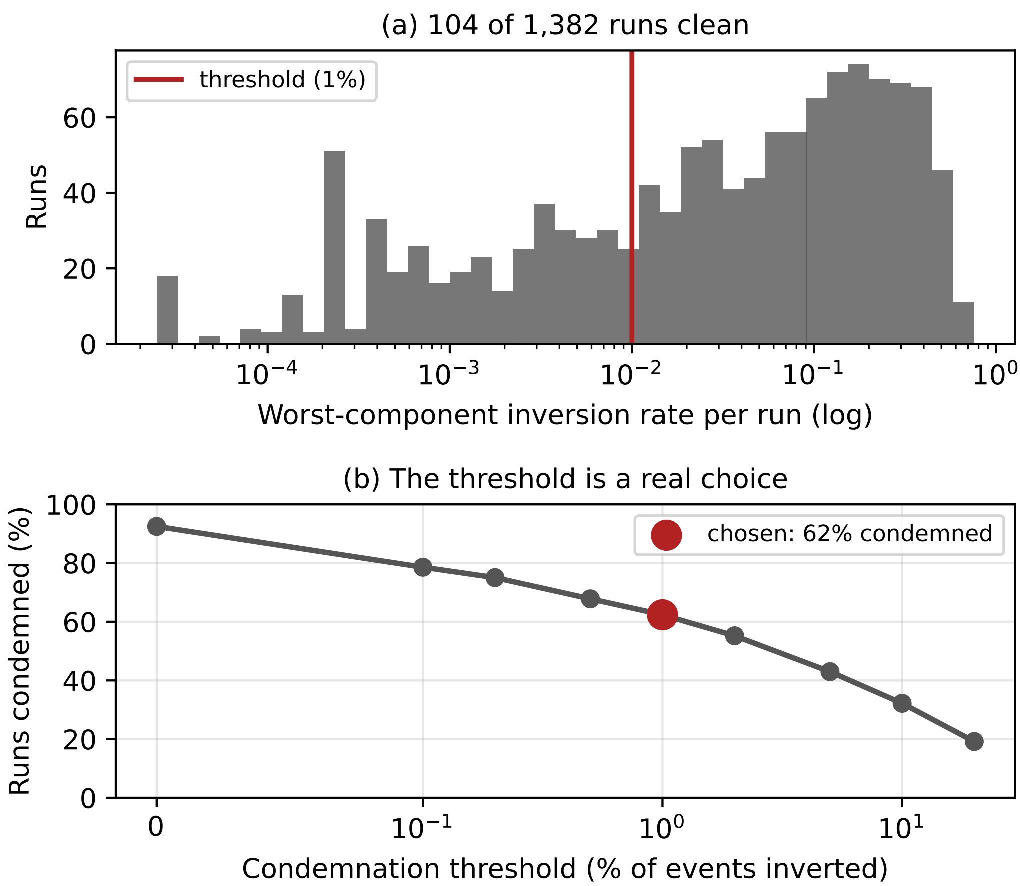

Fig. 4. The audit on Testbed A. (a) Per-run inversion rates: 104 runs are clean and the rest spread over three decades, so there is no natural threshold. (b) Share of runs rejected against the threshold. The choice matters for the run count and not for any conclusion.

6.5 A gap in our own provenance

Auditing the corpus turned the same scrutiny on our own record-keeping, and it did not survive it either. We report the finding here because the protocol in Section 8.2 is weaker without it.

Replay plans built from StatsBomb carry a **baked-in** 120× **time compression**: a plan's emission offset is already the match time divided by 120. The producer's `--speedup` flag then multiplies that. So `--speedup 1` against such a plan is *not* real time, it is 120×; true real time needs 1/120. The same flag means different things depending on which plan it is pointed at, and getting it wrong is silent: the run completes, the wall time looks unremarkable, and the numbers stay plausible. We lost a campaign to it before adding a pre-flight that derives the value from the plan and then checks the achieved rate against elapsed wall time.

The part we cannot repair is retrospective. **No surviving artefact records the achieved replay rate of any run we report.** Several superseded Testbed A scenarios record the nominal `--speedup` *flag*, which is not the same thing: converting a flag to a rate requires the plan's compression, and every plan those files name has since been deleted. Everywhere else the orchestrator wrote `speedup` into a per-campaign summary that lived beside the raw per-run data, and for the earliest corpora only the aggregated CSVs were kept. For the E1 corpus of Section 7.4 the surviving evidence is contradictory: the commit that introduced it states the runs were made at true real time, the campaign script reconstructed afterwards specifies `--speedup 10`, and the flag's meaning was corrected 21 hours later the same day. E1 was replayed at 1×, 120× or 1200×, and we cannot say which.

The rate is recoverable from the measurements, if not from the metadata. Having argued all paper that a physical constraint can decide what record-keeping cannot, we applied the same move here, and it works. The start-up cost of Section 7.5 is a clock: it lasts a fixed 102.6 ms of *wall* time, so the number of events it swallows depends directly on the replay rate. Counting how many events per run carry it therefore reads the rate back out of the data.

The E1 runs sort into cells by how many events they matched, and one cell is diagnostic. Fifteen Kafka runs matched exactly eight events, and their median scheduling lag is **52.34** ms (range 51.60–53.01) — not 103 ms, and not 1.6 ms. A median of eight values is the mean of the fourth and fifth. The only way to land midway between the two modes is for exactly four of the eight to be late, so the fourth value is a fast event and the fifth a slow one: $(1.59 + 103.5)/2 = 52.6$ ms against 52.34 observed. Four is also exactly what the loop trace shows at a verified real-time rate (Table 20), and it is what the plan predicts, since five events share t_{sim}=0 and the first of them pays the block rather than inheriting it.

That cell excludes the accelerated alternatives outright. At 120× the prologue spans 12.3 s of match time and holds 16 events; at 1200×, 123 s and 112 events. Under either, all eight matched events would be inside it and the cell would read 103 ms. It reads half that. **E1 was replayed at true real time**, the commit record was right, and the campaign script reconstructed afterwards describes a different, earlier campaign. The remaining cells agree: runs matching five or seven events read 103 ms, as four late events out of five or seven must give.

The inference has since been checked against the original script, and it holds. Archiving the cloud driver before releasing it turned up the campaign as actually invoked, in a shell script in the operator's home directory that no part of the repository had ever referenced. It specifies `--speedup 0.008333` — 1/120, against a plan already compressed 120×, which is true real time — and its log's first line, `E1 N=1 reps=8 00:39:40`, names the run directory the E1 artefact's first row still points at. The arithmetic above and the flag agree.

We report that as a check rather than as the answer, because the order matters. The rate was determined from the measurements while the metadata was genuinely unavailable, and the script surfaced afterwards, by luck, from a machine that was about to be switched off. What the agreement establishes is not that E1's provenance was fine all along — it was not — but that the recovery method returns the right answer when there is an answer to check it against. That is the one thing a physical-constraint argument can rarely demonstrate about itself.

The same script also explains a feature we had described and not accounted for. It passes `--max-t-sim 2`: two seconds of match time per run. The median of seven matched events that makes E1 under-powered, and that Section 7.5 treats as an opening burst, is that window. The burst is real and is why those seven events are the densest in the match; the reason there were only seven is that the campaign asked for two seconds.

We leave the disclosure standing rather than deleting it, for two reasons. The recovery was available only because the artefact we were chasing happened to be a wall-clock constant with a sharp signature, and the confirmation only because a virtual machine had not yet been destroyed; a reader should not have to reconstruct an experimental parameter by arithmetic on its results, and next time there may be nothing to reconstruct it from and nothing left to check it against. And the exercise makes the protocol rule concrete: what was missing was one number, written down once.

What the gap touched. The audit is unaffected either way: a negative transport is causally impossible at any replay rate, and the rejection counts are arithmetic on surviving per-event data. The withdrawal in Section 7.5 is unaffected by construction — we give the argument in a form that holds at every candidate rate. The rules of Section 7.1 were measured after the pre-flight existed, at a derived and verified rate. Section 7.4's external validity is restored by the recovery above, and the recovered script now documents it as well — though the inference stood on its own before the script appeared, and is what we would still be relying on had the machine been released a day earlier. Section 8.2 carries the rule that would have made all of this unnecessary.

6.6 The property that makes this general

One feature transfers beyond this study. The check tests against zero, so it rejects a measurement in proportion to how close that measurement sits to zero. Our network-delay arm, where Redis transport is tens of *seconds*, passes at 15 runs out of 15 at every injected delay — on the same hardware, in the same campaign, minutes apart from conditions that fail outright. A few milliseconds of timestamping race is invisible against a 31-second effect and fatal against a 0.34-millisecond one.

The check therefore bites hardest exactly where the scientific question is most delicate. This inverts a common intuition. In cross-process delay measurement, an effect of the same order as the instrument's own noise floor is the one most likely to have been manufactured by it, and statistical significance offers no protection because the artefact is systematic.

6.7 The same exposure in a benchmark we did not write

Everything above concerns our own corpus, which invites the obvious objection: we have shown that *we* made a mistake, not that anyone else is exposed to it. So we audited a harness we did not write — the OpenMessaging Benchmark [24], the most widely used cross-broker benchmark and one we cite in Section 2 — for the two properties that decide whether this failure can occur and whether anyone would see it. We report the audit at source level, with file and line, so a reader can check it against the upstream commit rather than take our word for it (commit `5b1fa70`, 7 April 2026).

It computes the same quantity the same way. In `LocalWorker.java`:294 the end-to-end latency is `now - publishTimestamp`, where `now` is a clock read in the *consumer* process and `publishTimestamp` arrives with the message. In the Kafka driver that timestamp is `record.timestamp()`, which under Kafka's default `CreateTime` semantics is the one written by the *producer* [1] — and therefore, in a distributed deployment, on the producer's host. The benchmark does not override that default anywhere in its configuration. The subtraction therefore spans two processes and, in a distributed deployment, two machines' clocks — the same construction that produced 1,321 impossible runs here.

And it discards the evidence. In `WorkerStats.java`:95 the sample is admitted to the histogram only `if (endToEndLatencyMicros > 0)`. The message is still counted as received one line earlier, but a non-positive latency is dropped, and no counter anywhere in the harness records how many were dropped.

We think this guard is defensive rather than evasive, and that is what makes it worth reporting. The recorder is an `HdrHistogram Recorder`, which rejects negative values, so a programmer who has seen one crash learns to filter for positives before recording. It is the reasonable local fix. Its non-local consequence is that the one observation which would prove the instrument had failed is the exact observation the guard removes — and because the filter is undocumented and uncounted, nothing downstream can tell that it fired. We are not describing carelessness. We are describing a defensive reflex that, applied to a cross-process span, quietly converts a detectable failure into an invisible one. Two consequences follow. The reported latency distribution is *conditioned on being positive*, so a causality violation cannot appear in the output even in principle. And the retention rate — the statistic we argue belongs beside every result, the way a survey reports its response rate — is not merely unpublished but unrecoverable from a completed run.

A third consequence is arithmetic rather than causal, and it bites at the fast end. `System.currentTimeMillis()` has millisecond resolution, so any delivery faster than one millisecond differences to exactly zero and is discarded by the same guard. On the co-located paths where such benchmarks are usually run — ours measures broker transport at 0.1 to 0.5 ms — that is not an edge case but a large share of the samples, and the reported distribution is truncated from below at 1 ms.

We then ran it, 80 *times, and the answer was not the one we expected.* A source audit is not a measurement, so we made the discards observable (Section 4.8) and swept the benchmark across background load, message size, producer rate and replication: 80 instrumented runs on the co-located path, each three minutes at rates around 500 messages per second.

6.7.1 The claim we withdraw, and where its refutation was. An earlier version of this section reported that a single such run discarded 6,000 samples under load, and read those discards as the same causality violation we report in Section 4.5. **That reading is withdrawn.** The counter behind it was one unsigned total, and both a causality violation and a sub-millisecond delivery fail a `> 0` test; counting them together cannot distinguish them. Separating them by sign across all 80 runs gives, in roughly 420,000 discarded samples, **not one negative**. The most negative end-to-end latency observed at any load, any message size and any producer rate is 0 μs.

We report where the refuting evidence was, because the answer is uncomfortable and is the paper's own subject one level up. It was in our artefact from the day we committed it. The counter printed one line per thousand discards, and each line carried the sample that triggered it; all eleven lines committed with that first run read `sample_micros=0`. Beside them, the benchmark's own progress output reported a median publish latency of 0.4–0.5 ms — sub-millisecond, which is precisely the condition under which a millisecond-resolution difference collapses to zero. The number that refutes the reading, and the mechanism that explains it, were both printed, committed, and read past. What was missing was not data but a reason to look at the sign, and the statistic we had chosen to report did not have one.

6.7.2 What the benchmark actually does. Across the 49 runs on an unsaturated path, the benchmark computes its reported latency distribution from between 0.36% **and** 100% of the samples it takes. Over that 278-fold range, its reported median takes **two values**: 1.0 and 2.0 ms. One run computed its summary from 998 samples and another from 120,425; both report a median of 1.0 ms, and nothing in the output distinguishes them.

This is the second failure mode in its plainest form. It is not that the benchmark is noisy. It is that the one statistic a reader consults is insensitive to a 278-fold change in the evidence behind it, while the quantity that would reveal the problem — how much data survived — is not reported at all and is unrecoverable from a completed run.

The benchmark's own output corroborates this without any instrumentation from us. Across eight runs it reports 32 percentile values, and *every one* is a whole number of milliseconds. The only fractional statistics it reports are the averages, which is the one column that could be fractional, being a mean of integers. Three runs report p50 = p95 = p99 = max = 1.0. That is not a narrow latency distribution; it is a distribution with a single value in it, printed to three decimal places.

Table 7. Retention against the producer's send interval, with message size, background load, duration and host held fixed. Only the commensurate rate is unstable, and the two incommensurate rates agree with each other despite intervals differing by 19%. Predictions were recorded before the runs.

Rate	Interval	Multiple of tick?	Retention per replicate	Spread
500/s	2.000 ms	**exact**	0.47, 1.51, 18.02, 99.99	99.5
457/s	2.188 ms	no	48.77, 49.50, 49.69, 50.87	2.1
383/s	2.611 ms	no	50.28, 51.46, 53.13	2.8

6.7.3 The mechanism: phase, not speed. Retention varies enormously, and the obvious explanation — that the path was faster in some runs than others — is wrong. OMB's own publish latency is measured within a single process and is *not* quantised to the millisecond grid, making it a clean probe of how fast the path actually was. Across the 49 unsaturated runs it spans 0.3 to 0.4 ms while retention spans 278-fold, at a rank correlation of +0.075. The path did not change.

What changes is *phase*. A sample survives only if its delivery crosses a millisecond boundary, which depends on where within the millisecond it was sent. OMB paces its producer at a fixed interval. When that interval is an exact multiple of the timestamp quantum, every sample in a run occupies the same phase, so either nearly all of them cross a boundary or nearly none do.

We establish this by manipulation, holding message size, background load, duration and host fixed and varying only the producer's interval relative to the millisecond grid (Table 7). At 500 msg/s — exactly 2.000 ms — retention across replicates spans 99.5 points. At 457 and 383 msg/s, incommensurate with the grid, it is stable to 2.1 and 2.8 points.

The stronger result is that the two incommensurate arms agree with *each other*. Their intervals differ by 19% and their median retentions by under two points. Had retention been a function of the pacing interval, they would separate; under a phase account the interval decides only *whether* samples dephase, not what fraction then survives, so they had to agree.

A quantitative form, derived and then measured. If send phases are uniform within the tick, a sample survives exactly when its delivery crosses a boundary, which for a true latency T_{true} below one tick τ occurs with probability T_{true}/τ. Hence

$$\text{retention} \;=\; \min\left(1,\ \frac{T_{\mathrm{true}}}{\tau}\right) \qquad \text{(dephased sampling).} \tag{3}$$

Both incommensurate arms sit near 50%, which with $\tau = 1$ ms implies $T_{\mathrm{true}} \approx 0.5$ ms — agreeing with the unquantised publish-latency probe above and with this project's own transport measurements of 0.1 to 0.5 ms. Three routes to one number is suggestive, but it is one point on a line, so we tested the equation by moving T_{true} while holding the rate incommensurate (Table 8).

Retention rises 33.3 points across the sweep, against a largest within-level spread of 2.6 points. Inverting Equation 3 gives an implied T_{true} per level, and at the two payloads large enough for serialisation to be measurable that implied latency rises *linearly* with payload: the increment per byte is 0.630 at 32 KB and 0.638 at 64 KB, two independent estimates agreeing to 1.3% and corresponding to an effective path of about 1.6 Gb/s.

We report the two smallest steps as uninformative rather than omitting them. At 1 and 2 KB the payload differs from baseline by under 2 KB, so the predicted change in T_{true} is smaller than the replicate noise; dividing that by the payload delta yields −2.600 and 0.908, which is scatter amplified by a small denominator and not evidence against the relation. The sweep discriminates only where the manipulation is large enough to exceed the noise, which is a property of our chosen

Table 8. Retention against payload at 457 msg/s, incommensurate with the millisecond grid so samples stay dephased. Implied T_{true} inverts Equation 3. The final column is the increment in implied latency per byte of added payload, relative to the 200 B baseline; the two smallest steps are noise-dominated and marked.

Payload	Retention (3 reps)	Median	Implied T_{true}	ΔT/byte
200 B	51.08, 52.03, 52.88	52.03%	0.520 ms	baseline
1 KB	50.11, 50.32, 51.47	50.32%	0.503 ms	−2.600†
2 KB	52.13, 54.62	53.37%	0.534 ms	0.908†
32 KB	68.01, 68.44, 69.40	68.44%	0.684 ms	**0.630**
64 KB	84.21, 85.36, 86.82	85.36%	0.854 ms	**0.638**

† payload delta under 2 KB; the predicted change is below replicate noise, so the ratio is scatter over a small denominator.

sizes rather than of the law — our first attempt at this sweep used sizes that were all in that regime and settled nothing.

6.7.4 Three replicates do not establish reproducibility. We ran the identical load sweep three times over nine hours. Per-level median retention reproduced at **one load level in five**, moving by 54 to 98 points at the other four. At 0% background load the three-replicate median was 1.51% in the first pass and 99.98% in the second.

The instructive detail is that agreement *within* a pass is no protection. In one pass the three replicates at 0% load agreed to 3.58 points — a tight, confident-looking measurement — and sat 98 points from what the same configuration produced hours earlier. Averaging replicates defends against noise around a stable value; at a configuration near the tick, retention has no stable value to be noisy around. Eighteen runs at one identical configuration gave six below 2%, eight above 90%, and four in between: their median is 23.4%, a value one run in eighteen produced.

6.7.5 What we cannot show, after eight attempts. The cross-host case is the one in which OMB's subtraction spans two clocks, and therefore the one in which its guard could discard a genuine causality violation. **We cannot report a measurement of it.** Across eight attempts at three background-load levels — including none at all, which refutes our own hypothesis that CPU starvation was the cause — the benchmark's distributed mode never completed a run, failing each time inside its own coordinator-to-worker protocol rather than in our instrumentation or configuration. The validity gate wrote `valid=0` with a reason on all eight, and those rows are in the artefact.

The exposure in that mode is therefore established by source audit — the timestamp is written on the producer's host and read on the consumer's, and the guard drops whatever the difference produces — and *not* by observation. We distinguish the two rather than let the audit stand in for a measurement.

What we do not claim. That any published result obtained with this benchmark is wrong. We have shown that the deletion happens, that it is large, that it is irreproducible, and that it is unreported. Whether it has affected a specific published measurement depends on that measurement's path latency and producer rate, and we have audited none. That asymmetry is the reason we recommend publishing a retention rate as routine practice: its cost is one number, and the alternative is a benchmark that cannot report its own failure.

6.8 What we withdraw

We withdraw the entire Testbed A arm, and on Testbed B the accelerated concurrency sweep, the connection sweep above ten connections, and the three-node cluster arm, which fails on every run in both systems.

A manipulation that did not manipulate, and what it cost. We also withhold a co-location campaign (E-A8). The intent was to shorten T_{true} by removing the network hop — the opposite of the payload sweep, and a useful check that the T_{true} result is not an artefact of how we lengthened it. It did not work. Moving the broker onto the driver made transport *longer* at idle, 0.512 → 0.573 ms, because it traded network latency for CPU contention on the same cores; under 88% load it moved transport by 1.03×, which is no manipulation at all. The inversion rates duly failed to separate (intervals overlapping in both cells). Reporting those rates as a null would misdescribe the experiment: nothing was manipulated, so nothing was tested. We record it because the pattern is itself informative. Two of our three attempts on this axis failed for structurally similar reasons — netem delayed both paths and cancelled in the difference, co-location traded one delay for another — which suggests T_{true} is a quantity that resists manipulation in a shared-resource system, because every route to shortening it lengthens something else. The payload sweep worked because padding adds serialisation without contending for the scheduler, and we were fortunate that it does.

7 What Survives

All results below come from Testbed B and pass the check. Each states its retention.

We order this section by consequence rather than by chronology. The rules of the failure model (Section 7.1) and the shape of Δ (Section 7.2) come first, because they are what transfers to a benchmark that is not ours. The answer to the question we originally asked (Section 7.4) comes after, because it is the narrowest thing here: it holds for two brokers on one workload, and the honest version of it is modest. The second withdrawal (Section 7.5) follows the broker result it corrects.

7.1 The model's rules, measured

Section 4.6 derived $\Pr[\text{inversion}] = F_{\Delta}(-T_{\text{true}})$ — the effect-size rule itself — and three further consequences, on utilisation, process count and instrument asymmetry. Each was pre-registered with its falsification criterion before the data existed, and all four are tested here on conditions that pass the check.

H1, the effect-size rule — on one contrast, because the other manipulation does not work. The evidence is the widest contrast, and it is clean: the co-located arm measuring $T_{\text{true}} \approx 0.5$ ms fails wholesale, while the network arm measuring tens of seconds passes every run on the same hardware (Section 6.6). Five orders of magnitude in the measured quantity, one instrument, opposite outcomes. **Supported.**

We withdraw the intermediate points, and the reason is worth more than they were. An earlier version supported H1 additionally with a `netem` delay sweep, reporting a rank correlation across injected delays of 0 to 50 ms. We then re-ran that sweep at a verified real-time rate with a manipulation check, and the check failed — not because the treatment was too weak, but because it does not act on the quantity at all. Injecting delay at the broker moves end-to-end TTI almost exactly as commanded (3.72 → 23.61 ms across 0 → 20 ms) while broker transport stays flat (0.535 → 0.482 ms; Table 9).

The cause is that the delay is **common-mode**. It delays the acknowledgement travelling broker-to-producer and the record travelling broker-to-consumer by the same amount, and transport is the *difference* of stamps taken at those two arrivals, so the injected delay cancels exactly. A

Table 9. Why the delay sweep is not an effect-size manipulation. Injected one-way delay at the broker, verified present at the interface, at a verified true real-time rate. TTI tracks the injection; broker transport does not move, because the same delay is added to both arrivals whose difference defines it.

Injected delay	TTI median	Transport median	Transport IQR
0 ms	3.72 ms	0.535 ms	0.162 ms
1 ms	4.67 ms	0.508 ms	0.166 ms
5 ms	8.66 ms	0.494 ms	0.159 ms
20 ms	23.61 ms	0.480 ms	0.150 ms

manipulation can be applied correctly, verified present at the interface, and still not touch the quantity under study — which is the same lesson as the rest of this paper, arriving this time in our own experimental design rather than in our instrument. The original sweep's correlation was therefore never an effect-size relationship, and we report the sweep only as the negative result it is.

H2, the utilisation rule — with the functional form withdrawn. Sweeping background load from idle to oversubscription, with no core pinning so the measured utilisation is the right denominator, inversion rate is flat below half utilisation and then climbs steeply (Table 11). The rank correlation is $\rho_s = 0.98$ ($n = 9$). By the pre-registered criterion — the M/G/1 form must fit better than a linear alternative — the rule passes comfortably ($R^2 = 0.945$ against 0.640), and we report that verdict as it was specified. **Supported.**

But the functional form does not survive a fairer test, and we withdraw it. Beating a straight line is close to guaranteed for any function that turns upward near saturation, so it is a weak discriminator. Refitting against convex alternatives whose exponents are themselves fitted — a power law ρ^k and an exponential $e^{k\rho}$ — an *exponential fits this data slightly better than M/G/1 does* ($R^2 = 0.961$ against 0.945). On the independent nine-level sweep of Section 7.2 the ordering reverses (0.958 against 0.897). Two campaigns disagree about which convex form is best, and the margins are of the same size as the disagreement, which is the signature of data that cannot discriminate between them at all.

What the data supports is therefore the *shape* and not the *mechanism*: inversion rate grows superlinearly in utilisation with a knee near saturation. It does not single out queueing theory. We had claimed the M/G/1 form specifically because we had only tested it against a straight line; that claim is withdrawn, and Pollaczek–Khinchine remains in Section 4.6 as the motivation for the prediction rather than as a fitted result. Distinguishing the forms would need points spaced to separate a pole at $\rho = 1$ from smooth exponential growth, which our nine levels are not.

H4, the oversubscription rule. At constant aggregate event rate, inversion rate rises with the number of concurrent producer/consumer processes: 0.029, 0.082, 0.105, 0.103 at $N = 1, 3, 6, 12$ ($\rho_s = +0.80$, $n = 4$). It is process count, not offered load, that drives the failure. **Supported.**

H3, the asymmetry rule. The model says that what biases a *comparison* is $\mathrm{E}[\Delta]$, and that it is non-zero only when the two endpoints stamp differently. Our two systems do: Redis stamps the acknowledgement on the calling thread the instant `XADD` returns, while Kafka stamps it in a delivery callback that runs on the client's I/O thread and must first be scheduled. So the prediction is specific — making Kafka stamp the same way Redis does should shrink the between-system gap, and shrink it from Kafka's side.

Two earlier attempts did not test this: both compared Kafka's two client libraries, and both of those stamp in callbacks, so the symmetric condition was never created, and we treat them as

Table 10. H3: median broker transport under the two stamping modes, both arms at one in-flight request under matched load. `callback` is the default asymmetric instrument; `inline` stamps Kafka on the calling thread as Redis already does. The between-system gap shrinks, and the shrinkage is entirely on Kafka's side.

Stamp	Kafka (ms)	Redis (ms)	Difference (ms)
`callback` (asymmetric)	0.392	0.106	+0.286
`inline` (symmetric)	0.322	0.107	+0.215

Table 11. H2: inversion rate against achieved CPU utilisation, no core pinning, background load swept from idle to oversubscription. The M/G/1 form fits at $R^2 = 0.945$ against 0.640 for a straight line.

Utilisation ρ	Inversion rate
0.003	0.007
0.253	0.009
0.504	0.022
0.628	0.047
0.753	0.132
0.878	0.207
1.000	0.208
1.000	0.221
1.000	0.264

untested rather than as evidence. Here we add an inline-stamping mode to the Kafka producer that stamps on the calling thread the moment the send resolves — structurally what `XADD` does — and run it against Redis under matched load, both at one in-flight request (Table 10). The gap falls from +0.286 to +0.215 ms, a 25% reduction, and the 0.071 ms it loses comes entirely from Kafka's side: Kafka's median transport drops from 0.392 to 0.322 ms while Redis's holds at 0.106. Removing the callback removed a 0.07 ms systematic offset that was being charged to the broker rather than to the instrument. An independent replication with half again as many runs reproduces it: the gap falls from +0.276 to +0.237 ms, again entirely on Kafka's side. **Supported.**

This is the rule with the most direct bearing on the paper's own first result. The spurious effect the audit rejected was a between-system difference; H3 shows that even after the audit, an asymmetric stamp leaves a smaller one behind, in the same direction, on the same pair of systems. An equal instrument is not a nicety.

7.2 The shape of Δ: a mixture, not a scale family

Equation 2 treats Δ as a single distribution. That is a leading-order picture, and a dedicated campaign — nine load levels from idle to oversubscription, 25 runs each, pre-registered before it ran — shows what it omits. We asked the strongest form of the model: if Δ were a scale family, differing across load only by a scale $\sigma(\rho)$, then inversion probability would depend on the measured quantity only through the standardised distance $z = T_{\text{true}}/\sigma$, and every condition's standardised tail would fall on one curve. It does not. At a matched $z \approx 3$ the tail mass ranges over 23× across load levels, far outside sampling error. **The scale-family hypothesis is falsified**, as we pre-registered we expected it would be.

What replaces it is a mixture, and the same campaign measures its two parts separately (Table 12). From idle to the knee the *core* of the distribution — its inter-quartile width, the ordinary stamping

Table 12. The mixture, measured across nine load levels (25 runs each, pre-registered). The core width σ_{core} = IQR/1.349 grows 5× from idle to the knee; the inversion rate — the tail — grows 60×. Inversions are temporally clustered at every load (runs-test z). The three $\rho = 1$ rows are distinct oversubscription levels that the utilisation coordinate cannot separate.

Utilisation ρ	Core σ_{core} (ms)	Inversion rate	Clustering z
0.003	0.126	0.004	−6.8
0.253	0.138	0.004	−6.8
0.504	0.180	0.006	−6.8
0.628	0.181	0.024	−6.9
0.753	0.238	0.076	−5.7
0.878	0.631	0.224	−4.9
1.000	1.224	0.371	−4.3
1.000	1.293	0.317	−4.7
1.000	1.710	0.261	−4.4

jitter — grows only 5×, while the *tail* that actually produces inversions grows 60×: a ratio of 12 to 1. Load barely widens the core; it multiplies the weight of a rare heavy tail. That tail is the descheduling event of Section 8.4: at every load, idle included, the inversions within a run are temporally clustered (Wald–Wolfowitz z from −6.9 to −4.3, all far below zero), so they arrive in bursts of consecutive events, not independently. The mixture unifies three otherwise loose observations — the clustering, the faster-than-quadratic growth of variance, and the failed collapse — under one mechanism: Δ is a narrow jitter core plus a rare descheduling tail whose *weight*, not width, load controls.

This refines rather than unseats the rules. H1, H2 and H4 all concern the tail's growth and stand; what the mixture adds is that a benchmark's risk near the noise floor is governed by that tail, so the single-Δ reading of Equation 2 *understates* risk at low load, where the core is tight but the tail is already non-empty.

Recovering the tail without a reference clock, and where it reproduces. Because inversion rate at a threshold *is* a quantile of Δ's tail, the tail can be read off inversion counts alone — an instrument-quality estimate needing no reference clock. The honest test of it is not internal consistency but reproduction: do the recovered tail quantiles agree between two independent campaigns at matched utilisation? Below saturation they do — 12 of 17 matched quantiles overlap at 90%, and every one of the deep pre-knee quantiles does. The failures are all at $\rho = 1$, where the coordinate is degenerate — eight, ten and twelve background workers all read as full utilisation while representing different degrees of overload — so the two campaigns are not in fact at matched load there. The recovered curve reproduces wherever utilisation is a meaningful coordinate and stops when it stops being one, which is the correct boundary for the method rather than a defect in it.

7.3 What the mixture is: a rare state, not a wide distribution

Section 4.6 models Δ as one distribution. Three results have now cut against that: the scale family is falsified, the M/G/1 form is withdrawn, and the manipulation we used to probe the effect-size axis turns out not to act on it. This section states what we think replaces it. We separate carefully what the data forces from what we inferred by looking at it.

The observation no single distribution can produce. Inversions *cluster in time*: Wald–Wolfowitz z from −4.3 to −6.9 at every load, idle included (Table 12). They arrive in runs of consecutive events.

A distribution — of any shape, however heavy its tail — describes how *large* a delay is and says nothing about *which* events receive one. Clustering is a statement about time, and it is the fact the old model had no way to use.

The model. Suppose the stamping thread is in one of two states. When *running*, it reads the clock promptly and the delay is ordinary jitter: a narrow core of width σ_c. When *preempted*, it is off-CPU, and every event that becomes ready during that interval inherits the residual time until it is rescheduled. Writing p for the fraction of time preempted and $S(t)$ for the survival function of that residual,

$$\Pr[\text{inversion} \mid T_{\text{true}}] \;=\; p(\rho)\; S(T_{\text{true}}). \tag{4}$$

The failure is then a *rare state*, not a wide distribution. Load does not mainly stretch the delay; it changes how often the thread that must record the time is not running.

What it accounts for. Clustering follows by construction: being preempted is an *interval*, so the events inside it are corrupted together and a two-state process cannot help but cluster. The 60× growth in tail weight against 5× in core width is two different parameters moving — σ_c belongs to the running state, p to the scheduler. The scale family fails because a scale family has one parameter and here two move at very different rates.

A prediction that points the opposite way. The two structures disagree geometrically. A scale family changes σ, so its tail curves *rescale horizontally*; Equation 4 changes p, so its curves *shift vertically and stay parallel* on a log scale. Taking the log ratio of two conditions' tail masses at several thresholds, the two-state model predicts a constant, independent of the threshold.

Restricting to tail estimates backed by at least 20 events — nine far-tail points resting on five to eight events are excluded, as they carry log-scale errors larger than the effect — the median spread of that log ratio across conditions is **0.29**, a factor of 1.34, with a worst case of 0.91. Set against the 23× failure of the scale-family collapse, the tail curves are close to parallel. The two conditions with the largest spreads sit at intermediate load, where the residual distribution is plausibly still shifting, so the honest statement is $\Pr[\text{inv}] = p(\rho)\, S(c;\rho)$ with p varying fast and S slowly: separability is a good first-order description rather than an exact law.

What this is worth, and what it is not. It is mechanistic where the old account was descriptive — "risk rises with utilisation" is a correlation; "the stamping thread is preempted a $p(\rho)$ fraction of the time, and every event arriving in that window is corrupted together" names the moving part. It is also actionable in a way a distributional model is not: if the failure is occupancy, the mitigation is not a better clock but keeping the stamping thread runnable — a dedicated core, a raised priority, or a stamping path the workload cannot preempt.

But we found this by looking at the data. The separability test above is exploratory, and Section 8.5 says so. We therefore pre-registered a confirmatory version and ran it: the campaigns that follow in this section are that test, and the decisive one — raising the stamping threads to real-time priority at fixed utilisation — is reported immediately below. Two states remains the smallest model consistent with what we see, not a claim about what the kernel does.

On the load axis the model is a tautology, and we say so before using it. Read as an account of how the inversion rate depends on ρ, Equation 4 has no content. With p unconstrained, *any* monotone rate curve can be written as $p(\rho)\, S$ by setting $p = \text{rate}/S$. Its content is on the threshold axis, which is where we tested it above and where it passed. We are explicit about this because Section 7.1 withdraws the M/G/1 form — and, once the sweep reached the utilisation range where the candidate

Table 13. **Occupancy, not utilisation.** Stamping processes at ordinary priority versus `SCHED_FIFO`, with the background load unchanged. Utilisation is measured in both arms, not assumed: it matches to 0.001, so the manipulation moved occupancy alone. The inversion rate falls by a factor of 39–54 to the running-state floor. Utilisation-only models predict no change.

	Utilisation ρ		Inversion rate		
Load	ordinary	real-time	ordinary	real-time	factor
75%	0.7531	0.7525	0.1320	0.0034	39×
88%	0.8809	0.8812	0.3049	0.0057	54×

forms diverge, refutes it outright — so a replacement that cannot fail would be a worse error than the one it replaces.

Constraining p to a power law in ρ makes it testable, and it fits our ladder poorly ($R^2 = 0.65$ in log space). What fits is the form obtained by restoring the quantity the separability test divides out. That test standardises thresholds by σ; an inversion requires the residual to exceed T_{true}, so the standardised threshold is $T_{\text{true}}/\sigma(\rho)$, and since σ grows fivefold across the ladder the threshold slides toward zero as load rises. Both factors then climb with load: $\Pr[\text{inv}] = p(\rho)\, S(T_{\text{true}}/\sigma(\rho))$, which reaches $R^2 = 0.9905$ against 0.9811 for a fitted exponential in ρ and 0.8863 for $(\rho/(1-\rho))^k$, at three free parameters each.

That lead is not evidence, and we do not report it as a result. Freezing σ at its mean and refitting *improves* the fit, to $R^2 = 0.9982$. On our ladder σ rises monotonically with ρ: the two regressors move together, so no fit can credit one over the other, and the corrected form's advantage is bought by reading two more measured columns than its competitors are given. This is a limit of the experimental design, not of the analysis, and no further work on these data can lift it.

The experiment that settles it. The way out is to move one of the two and hold the other fixed. Raising the stamping processes to real-time priority makes them preempt the background load rather than queue behind it, so occupancy falls sharply while utilisation does not move — the same stressor is doing the same work. The two readings then separate by an order of magnitude: an occupancy mechanism predicts the inversion rate collapses toward the running-state floor, near 0.004, while any account in which the rate is a function of ρ predicts no change at all, because ρ is unchanged by construction.

This is also the manipulation our delay injection should have been. Injecting delay at the broker failed because it delayed the acknowledgement path and the delivery path equally and cancelled in the subtraction (Section 7.1, Table 9); scheduling priority has no such symmetry, because it acts on the stamping threads themselves, which is where the model locates the failure.

The result is unambiguous (Table 13). Utilisation held to within 0.001 at both levels, so the manipulation check — fixed in advance, and empowered to withhold the comparison — passes. The inversion rate fell from 0.132 to 0.0034 at $\rho = 0.75$ and from 0.305 to 0.0057 at $\rho = 0.88$: factors of 39 and 54, with disjoint Wilson intervals. The two residual rates land on the running-state floor the model names in advance, $C_0 \approx 0.004$.

This refutes every account in which the inversion rate is a function of utilisation. M/G/1 and a fitted exponential in ρ both predict no change here, because ρ did not change. The rate changed by a factor of fifty. Whatever the correct load model is, ρ is not its variable — and that single fact reorganises the rest of this section.

Both load-axis predictions failed, including ours. The knee sweep placed conditions at achieved ρ of 0.881, 0.920, 0.950, 0.970 and 0.990, where the candidate forms diverge most; the earlier ladder stopped at 0.878 and everything above it collapsed onto a degenerate $\rho = 1.000$, which is why Section 7.1 could not settle the question. With those points in hand, M/G/1 fits *worse than the mean* ($R^2 = -0.05$ against a fitted exponential's 0.93). The withdrawal does not merely stand: the forms are now separated, and M/G/1 is the one ruled out.

The bounded prediction fared better but also failed. Over $\rho = 0.881$ to 0.990 the rate grew by a factor of 1.44. M/G/1 predicted 13.8; the bracket we registered before the sweep predicted 2.45–3.07. The rate saturates, as a bounded mechanism requires — but harder than our own extrapolation allowed, and a prediction that misses low is still a prediction that missed. We report it as a failure rather than as the nearer of two misses.

The two failures have one cause, and E-A5 names it. Our bracket was still parameterised in ρ, through $p = \rho^C$; E-A5 shows ρ does not determine occupancy. What survives is the mechanism — established by manipulation — and not any curve in ρ, ours included.

Utilisation is not the variable, shown at identical utilisation. The cleanest demonstration does not need a curve at all. Two loads can reach the same ρ by different means: `--cpu k` runs k cores flat out and leaves $C - k$ genuinely free, while `--cpu C --cpu-load P` duty-cycles every core and leaves none. If the mechanism is whether the stamping thread can find a core, the second should be worse at equal ρ. We ran both in one campaign, interleaved, at utilisations quantised to k/C so the two arms could match (Table 14).

At $k = 6$ the achieved utilisation is 0.7531 in *both* arms — identical to four decimals — and the inversion rates differ by a factor of 2.07. A function of ρ returns one value for one input, so ρ cannot be the variable, and this is not an inference across campaigns run on different days: same machine, same protocol, same hour, only the arrangement of the load differing. Each cell rests on 2 985 matched events, so the separation is decisive on its own terms: a two-proportion test gives $z = 10.3$ at $k = 6$, and the Wilson intervals do not overlap.

The gap narrows sharply at $k = 7$ (1.06×), which is the mechanism showing itself rather than a weaker result: with seven of eight cores busy, one free core is nearly as useless as none.

We ran the whole campaign again, and it corrects us on one point. The replication (E-A6b, right-hand columns of Table 14) reproduces the load-bearing cell almost exactly: at $k = 6$ it again finds both arms at $\rho = 0.7531$ — identical to four decimals in all four arms across the two campaigns — and a ratio of 2.05× against the original 2.07×. The central claim is therefore replicated, at matched utilisation, on a different day.

Two cells behave less tidily and we report them rather than the average. At $k = 5$ the effect reproduces in direction but not in size (1.88×, then 4.61×), consistent with the day-to-day level drift we document for the 88% baseline. At $k = 7$ the convergence itself is what fails to settle: the first campaign found the two geometries statistically indistinguishable ($z = 1.22$, intervals overlapping), and on that basis an earlier draft of this section reported the convergence as reaching a null. The replication does not support that. It finds a small but clearly resolved difference (1.19×, $z = 3.46$, intervals disjoint), so we withdraw the stronger reading. What both campaigns agree on is the *shape*: a twofold separation at $k = 6$ collapsing to near-parity at $k = 7$. Whether it closes to exactly zero is not established by these data, and the mechanism does not require that it should — seven busy cores out of eight leave a little slack, not none.

The other side of the inequality, and it moves the rate the wrong way. Every manipulation so far acts on the stall distribution. The mechanism also predicts something about T_{true}, and that prediction is uncomfortable: a *slower* path should be a *more reliable* measurement, because the same stalls now have a longer interval to outrun.

Table 14. **Same utilisation, different rate.** Both load geometries in one campaign. Concentrated load leaves $C - k$ cores free; spread load duty-cycles all of them and leaves none. At $k = 6$ the two arms sit at the same ρ to four decimals — in *both* campaigns — and the rates differ twofold in both. Every cell is 2 985 matched events over 25 runs; z is a two-proportion test between the arms of that row. The $k = 7$ row is where the two campaigns disagree: the original cannot separate the geometries there, the replication can, and we withdraw the stronger reading (text).

	E-A6 (original)			E-A6b (replication)			
k/C	conc.	spread	z	conc.	spread	z	ratios
5/8	0.0201	0.0379	4.09	0.0111	0.0509	8.89	1.88×, 4.61×
6/8	0.0824	0.1709	10.27	0.0600	0.1229	8.44	2.07×, 2.05×
7/8	0.2281	0.2415	1.22	0.1973	0.2342	3.46	1.06×, 1.19×

Achieved ρ, $k = 6$: 0.7531 in all four arms. $k = 5$: 0.6284/0.6250 and 0.6284/0.6259. $k = 7$: 0.8778/0.8809 and 0.8778/0.8822.

Table 15. **A slower path is a more reliable measurement, twice.** Payload padding lengthens the true transport without touching the scheduler. Utilisation is held to a 0.012 spread; the serialisation cost pushes it slightly up, against the predicted direction. Both quantities move monotonically, in opposite directions, in both campaigns: transport spans 76.9× and 76.8×, the rate falls 4.09× and 4.26×. Each cell is 2 985 events.

	E-A10		E-A10b		
Padding (B)	Transport (ms)	Inversion	Transport (ms)	Inversion	ρ span
0	0.701	0.2603	0.761	0.2338	0.881–0.882
4 096	2.381	0.1906	2.474	0.1652	0.882–0.883
65 536	14.638	0.0888	16.059	0.0784	0.885–0.885
262 144	53.898	0.0637	58.482	0.0549	0.893–0.895

We lengthened T_{true} by padding the payload, which adds serialisation and transfer time without touching the scheduler (Table 15). Transport rose 76.9×, from 0.70 to 53.9 ms, and the inversion rate *fell* 4.1×, monotonically, with utilisation held to a 0.012 spread. Any account in which inversions track how hard the system is working predicts the opposite, since larger payloads are strictly more work — and the serialisation cost pushes ρ slightly *up* across the sweep, against the observed fall. The effect is measured against its own confound.

This also constrains the stall distribution sharply. A 77× increase in the threshold bought only a 4.1× reduction in rate, so $\Pr[\text{stall} > 53.9\text{ ms}]$ is merely four times smaller than $\Pr[\text{stall} > 0.70\text{ ms}]$. The tail barely thins across two orders of magnitude, which is why the mean-based counters above could not account for the effect and why the direct trace was necessary.

Closing the loop: measuring the stall distribution directly. Every result so far infers the scheduler's behaviour from the inversion rate it produces. The mechanism says something stronger and directly checkable — that the inversion rate should equal the probability that a run-queue stall outlasts the interval being measured — so we measured that probability without reference to the inversions at all. Using `bpftrace` on the `sched_wakeup` and `sched_switch` tracepoints, we recorded the delay between a stamping thread becoming runnable and reaching a core, for every such transition in the run, and computed $\Pr[\text{stall} > 0.5\text{ ms}]$ against the measured transport of that cell (Table 16).

The two quantities share no instrument, no estimator and no data: one is a count of negative transports in application CSVs, the other a kernel histogram of scheduler transitions. We have since run the campaign again at two load levels, so there are three ordinary arms rather than one, and they agree with their traced tails to within about 30% in every case: ratios of 0.78, 1.06 and 1.32 (Table 16), with nothing fitted between them.

The replication matters for more than the count. On the single arm we had first, the traced probability sat 22% *below* the observed rate, and we wrote that this was the opposite of what the simplest reading predicts — not every stall lands on a stamping instant, so the probability that one occurred should exceed the rate at which they corrupt a measurement. With three arms the residual has no consistent sign: it is 22% low at one, 6% high at another, 32% high at the third. What we reported as a directional puzzle was scatter. The honest statement is that a traced scheduler quantity predicts an application-level failure rate to within a third, unfitted, and that we cannot resolve any bias inside that.

The instrument check, and what it can and cannot rule out. Tracing a system to measure its scheduling behaviour is exactly the kind of intervention that can create what it observes, so the traced cell has an untraced twin. Our first attempt at this campaign ran with a probe that failed to attach: the histogram was empty, but the inversion rates are valid, and we kept that run as the control. It measured 0.2717 against the traced run's 0.2315 — the traced rate is 14.8% *lower*, two hours apart, same design.

We report that rather than a flattering comparison, because an earlier version of this section compared the traced cell against a different campaign's 88% arm (0.2214) and called the 4.6% gap a clean bill of health. The honest reading is weaker. The ordinary 88% arm has been measured five times across these campaigns and spans 0.221 to 0.305, a factor of 1.38; the traced value sits inside that spread, and so does its untraced twin. So we can say that tracing did not move the rate by more than the drift already present between campaigns, and we cannot say anything tighter than that. A tracing effect smaller than about 15% is not resolvable with this control.

What that limits is the *level*, which no claim here rests on. The comparison in Table 16 is between two quantities measured in the *same* run — a kernel histogram and a count of negative transports over the same events — so a common perturbation of that run moves both and leaves their agreement intact.

The gate fired on our own replication, and we report the arm it withheld. The check is a rule fixed in advance: a traced arm whose inversion rate differs from its untraced comparator by more than 25% is not compared. The replication's 88% arm measures 0.1956 against the same untraced control's 0.2717 — a drift of 28% — so its verdict is withheld by the same mechanism that admitted the original. Its numbers are in Table 16 because withholding a comparison is not a reason to hide a measurement, and because the ratio it would have contributed, 1.06, is the closest agreement in the table. Reporting only the arms that passed would leave a reader with three ratios and no idea that one of them is inadmissible under our own rule.

The real-time arm's zero is an artefact of the instrument, and the replication settles it. Every traced real-time arm we have run — three of them, across two campaigns and two load levels — recorded *zero* inversions in 2,985 events. The untraced twin of one of those arms recorded 15. Under that rate the chance of a single traced arm showing none is $3{\times}10^{-7}$, and of all three about 10^{-20}.

We first reported this as unresolved: either the arm genuinely differed or tracing perturbed it, and one measurement could not separate the two. Three can. Attaching BPF to `sched_switch` suppresses the inversions in the real-time arm, and the effect is not subtle — it removes all of them. The direction is consistent with the ordinary arm, where the two traced measurements are the lower two of the six we have at 88%, though there the drift between campaigns is wide enough that the same conclusion cannot be drawn.

This is the paper's own thesis arriving uninvited. We attached an instrument to measure why an instrument was failing, and the second instrument changed the thing it came to observe — in the

Table 16. **A kernel trace predicts the inversion rate, unfitted, in every arm that has one.** $\Pr[\text{stall} > T_{\text{true}}]$ from `sched_wakeup`/`sched_switch`, against the inversion rate measured from application timestamps in the *same* cell; the two share no instrument and no estimator. Every cell is 2 985 events. The three ordinary arms agree with their traced tails to within about a third, with no consistent sign. Every real-time arm reads exactly zero where the untraced control of the E-A9 pair reads 15/2985; that is a tracing artefact, not a floor (text). The E-A9b 88% arm is *withheld* by the instrument check — 28% drift against a 25% rule fixed in advance — and is shown because a withheld comparison is still a measurement.

Campaign	Arm	Load	ρ	Traced events	Pr[stall > 0.5 ms] / rate	Ratio
E-A9	ordinary	88%	0.8822	551,956	0.181 / 0.231	0.78
E-A9	real-time	88%	0.8819	570,591	0.018 / 0.000	—
E-A9b	ordinary	75%	0.7543	709,819	0.168 / 0.128	1.32
E-A9b	real-time	75%	0.7531	641,308	0.016 / 0.000	—
E-A9b	ordinary	88%	0.8825	687,986	0.207 / 0.196	*1.06*†
E-A9b	real-time	88%	0.8822	649,788	0.016 / 0.000	—
untraced control (E-A9's own first attempt, probe never attached)						
—	ordinary	88%	0.8812	—	— / 0.272	—
—	real-time	88%	0.8809	—	— / 0.005	—

† withheld by the instrument check; shown, not used.

arm where the quantity was smallest, which is exactly where Section 6.6 says a measurement is most exposed. It costs us nothing here, because no claim rests on the real-time arm's level, and it is worth stating plainly rather than leaving in a footnote: a kernel tracer is not a free observer.

A rule with no free load parameter, and what its exponent means. The payload sweep varies T_{true} at fixed load, so it probes the shape of the stall distribution directly rather than its response to stress. If run-queue delay has a heavy tail with index α, then $\Pr[\text{stall} > t] \sim Ct^{-\alpha}$, and the inversion rate should be a power law in T_{true} with no load term at all. Fitting the four points of Table 15:

$$\Pr[\text{inversion}] = 0.238\ T_{\text{true}}^{-0.339}, \qquad R^2 = 0.990 \text{ over a } 77\times \text{ span.} \tag{5}$$

The exponent is below one, so the stall distribution has neither a finite mean nor a finite variance. That is the part worth stating carefully, because it converts an observation into an explanation. Section 7.3 found that cumulative scheduler counters — means of occupancy and of stall length — moved by factors of two to five against a fortyfold change in the inversion rate. With $\alpha < 1$ that failure is not bad luck in the choice of counter. The sample mean of such a distribution does not converge as samples accumulate; it wanders with sample size. An instrument built on averages is therefore *structurally* unable to see this failure, not merely insensitive to it, and no amount of additional sampling repairs it.

The exponent replicates. E-A10b repeats the sweep end to end on a different day. Fitting its four points independently gives $\alpha = 0.344$ against the original 0.339 — agreement to 1.5% — with $R^2 = 0.995$. The prefactor moves more (0.238 against 0.216), which is what one expects when the level drifts and the shape does not, and it is the shape that carries the claim: α is below one in both campaigns, so both say the stall distribution has no finite mean. The rule therefore rests on two measurements rather than one.

Equation 5 and the kernel trace were produced by different campaigns, on different instruments, and can be checked against each other. The rule predicts $\Pr[\text{stall} > 0.5\,\text{ms}] = 0.301$; the trace measures 0.181. The ratio is 1.66 — the same order, from a fitted payload sweep and a `sched_switch` histogram sharing no data. Refitting on the replication gives 0.274 against the same traced 0.181, a ratio of 1.52: the gap narrows slightly and does not close. The rule over-predicts in both, which is the expected direction, since not every stall lands on a stamping instant.

We state the limits plainly. α is fitted, not derived, from four points per campaign at one load level, and the residual factor of about 1.6 between the rule and the trace is unexplained rather than accounted for. What the replication buys is that the exponent is a property of the system rather than of one afternoon's measurement; what it does not buy is a derivation, or an account of the gap.

Two bounds the rate respects, and where each comes from. The mechanism predicts a floor and the data supply a ceiling, and both are measured rather than fitted.

The floor is the idle rate. If inversions come from stalls, then a stamping thread that cannot be made to wait should invert at the rate of a machine with nothing else running. Two unrelated campaigns give the comparison: the real-time arms under load settle at 0.0049, and an *unloaded* host at ordinary priority measures 0.0035 — a ratio of 1.38. Heavy load with a runnable stamping thread looks, to this measurement, almost exactly like no load at all. The residue is not zero because a real-time thread still waits occasionally, which is why the model carries a floor term rather than predicting zero.

The ceiling is physical, and it is not one. Saturating the machine does not drive the inversion rate toward unity. It saturates near 0.371, and the three campaigns that reach saturation agree to within 1.11×. This matters for interpretation rather than for magnitude. Writing the rate as $P = p \cdot S$, where p is the probability that a stall is long enough and S the share of events exposed to one, a ceiling measured at 0.37 estimates S directly: even with every core saturated, roughly two events in three are stamped without ever being preempted. S is therefore a measured property of the workload's timing against the scheduler's, not a free asymptote fitted to make a curve close.

7.4 The brokers are equivalent within a millisecond, but not indistinguishable

What this claim rests on. We state the evidential basis first, because an earlier version of this section rested it on the wrong corpus. The load-bearing measurement is a powered replication at a replay rate that is *derived from the plan and verified against wall time before the campaign runs* — a documented setting, not an inferred one — over a median of 127 events per run. The original E1 corpus (Table 18) is reported afterwards, as the historical measurement it is, for two reasons that disqualify it from carrying the claim: its replay rate is an inference — recovered in Section 6.5 as true real time, but inferred from the runs rather than recorded — and it matched a *median of seven events per run*, the same seven-event opening burst as the scheduling lag we withdraw in Section 7.5. A sub-millisecond difference cannot be resolved on seven events, and E1 does not resolve one: its Hodges–Lehmann shifts wander (0.021, 0.116, 0.053 ms) and its N=1 estimators disagree. That E1 finds the two systems "equal" is as much a statement about seven events as about the brokers. What E1 does establish, and what the powered campaign confirms, is the *concurrency* result: Kruskal–Wallis finds no significant effect of concurrency in either backend ($p = 0.061$ for Kafka, $p = 0.091$ for Redis), and neither system degrades across the range.

The powered measurement. We measured transport directly, at a replay rate derived from the plan and verified against wall time, over a median of 127 events per run rather than seven, at $N \in \{1, 9, 12\}$ with 15 replicates each (Table 19). The picture is sharper and it is not the one E1 painted. Broker transport is Kafka ≈ 0.54 ms against Redis ≈ 0.11 ms: a Hodges–Lehmann shift of 0.41 ms, Kafka slower, with a bootstrap 90% interval of width ±0.006 ms and $p < 10^{-26}$ at every N. The two systems are *not* statistically indistinguishable; Redis's in-memory `XADD` really is several times faster per operation than Kafka's replicated-log append, which is the direction the grey-literature comparisons report.

And yet the same three estimators — Welch, percentile bootstrap and Hodges–Lehmann — agree that the difference is *equivalent within one millisecond* at every N, because 0.41 ms is comfortably inside that margin. Both statements are true and neither is idle: against the seconds-scale human annotation latency that dominates any live sports feed's end-to-end budget, a 0.41 ms broker difference is parts in 100,000, so for choosing a broker it is noise; but it is a real, tight, reproducible effect that a properly powered instrument resolves and a seven-event one cannot. The shift is also flat across concurrency (0.409, 0.418, 0.420 ms at $N = 1, 9, 12$), so neither system degrades. A part of even this residual is the instrument rather than the broker: Section 7.1 shows that 0.07 ms of the gap is the asymmetric acknowledgement stamp, leaving a true broker-transport difference near 0.34 ms. That 0.07 ms is a concrete instance of the blind spot of Section 4.7: it is a bias smaller than the quantity measured, so it never turns a value negative and the consistency check passed both corpora that carried it. The check guarantees a corpus is not *catastrophically* wrong; resolving a 0.07 ms instrument artefact still took a controlled experiment.

An independent replication confirms it. A second powered campaign, run days apart with the same protocol, gives Hodges–Lehmann shifts of 0.397, 0.412 and 0.413 ms at $N = 1, 9, 12$ — inside the 90% interval of the first campaign at every level, with overlapping intervals throughout. The 0.41 ms advantage is not a fluctuation of one campaign.

Why E1 saw near-equality, tested directly. The two campaigns disagree by more than power: E1 puts both systems near 0.8 ms and within 0.1 ms of each other, while the powered campaign puts them at 0.53 and 0.11 ms and 0.41 ms apart. Attributing that to seven events is an explanation only if it survives being checked, so we checked it. If the difference is the observation window and nothing else, then restricting the *powered* runs to their first seven events — in emission order, the same events E1 would have caught — must reproduce E1, and the same runs must give the powered answer over their full length. That is a prediction with one free parameter fixed in advance, and it can fail.

It does not fail (Table 17). Over all events the runs give shifts of 0.381, 0.417 and 0.414 ms, reproducing the powered 0.41 ms. Over their first seven they give 0.088, 0.129 and 0.248 ms — E1's wandering near-equality, and of the same magnitude as E1's own 0.021, 0.116 and 0.053 ms. The *absolute* medians match as well, which the shift alone does not force: the prologue puts Kafka at 0.83–1.02 ms and Redis at 0.74–0.81 ms, against E1's 0.79–1.00 and 0.72–0.86. One set of runs, one rate, one protocol, two windows, both published answers.

So E1 is not wrong and the campaigns do not contradict each other. E1 measured the opening burst, during which both systems are slow for the reason Section 7.5 establishes — topic auto-creation, connection setup, and the first blocking `produce()` — and that shared start-up cost swamps a 0.41 ms difference. As the run lengthens the cost dilutes, and the brokers' steady-state separation emerges. The seven-event corpus answered a question about start-up while appearing to answer one about brokers, which is the same failure of window selection as Section 7.5, one level up.

This refines E1 rather than contradicting it, and it sharpens the contradiction with Table 4: the accelerated corpus reported Redis *degrading* with concurrency; the powered real-time measurement finds Redis uniformly *faster* and flat. The reversal is the audit's doing, and the measurement's.

7.5 The end-to-end difference, and why we withdraw it too

TTI and transport give opposite answers in Table 18. Of Kafka's 105.5 ms, 102.9 ms is producer scheduling lag against Redis's 1.8 ms, while broker transport is 0.84 and 0.80 ms. An earlier version

Table 17. The same runs reproduce both published answers. Transport medians (ms) from the powered campaign, computed over every event and over each run's first seven events in emission order — the window E1 saw. The all-events columns reproduce the powered 0.41 ms shift; the prologue columns reproduce E1's near-equality (Table 18) in both shift and absolute level. The window, not the brokers, accounts for the disagreement.

		All events			First 7 (prologue)		
N	Runs	Kafka	Redis	Shift	Kafka	Redis	Shift
1	5	0.476	0.095	0.381	0.831	0.743	0.088
9	45	0.533	0.116	0.417	0.939	0.810	0.129
12	55	0.527	0.113	0.414	1.018	0.769	0.248

Table 18. **Historical corpus; the scheduling-lag columns are withdrawn.** End-to-end delay and its decomposition, Testbed B, after the consistency check (164 of 201 runs retained). Medians, ms. **The TTI and scheduling-lag columns record a per-run start-up cost, not a per-event property, and are retained only as evidence for the withdrawal in Section 7.5; they should not be read as system behaviour.** The transport columns are computed over the same seven-event opening burst, and so measure start-up rather than steady state: Table 17 reproduces them from the powered runs by restricting those runs to the same window. For steady-state transport see Table 19. The replay rate was not recorded and is recovered as true real time in Section 6.5.

	TTI		Sched. lag		Transport	
N	Kafka	Redis	Kafka	Redis	Kafka	Redis
1	105.3	5.0	103.0	1.4	0.792	0.721
9	105.5	5.4	103.0	1.9	0.802	0.807
10	105.3	5.8	102.8	2.0	1.000	0.855
12	105.7	5.3	102.9	1.7	0.839	0.844

Table 19. Powered transport replication at a verified true-real-time rate, over a median of 127 events per run (not the seven of Table 18), 15 replicates per cell. Medians in ms; the Hodges–Lehmann shift is $Kafka - Redis$ with a percentile-bootstrap 90% interval. The brokers are equivalent within 1 ms at every N (all three estimators) yet cleanly distinguishable: Redis is $\approx$ 0.41 ms faster, and the shift does not grow with concurrency.

N	Events/run	Kafka	Redis	HL shift [90% CI]
1	148	0.512	0.099	0.409 [0.394, 0.421]
9	121	0.539	0.115	0.418 [0.412, 0.424]
12	127	0.540	0.114	0.420 [0.414, 0.425]

of this paper reported that twentyfold end-to-end gap as a finding, attributed it to the client's sender thread waking from idle, and built a practitioner recommendation on it.

That gap does not survive re-measurement, and the reason is a second instance of this paper's own thesis. We report it at length for the same reason we reported the first, and we retain the figure that once carried it (Figure 6) rather than deleting the evidence.

The offset is a start-up cost, not a per-event constant. Re-measuring at N=1 on the same driver and the same broker, at a replay rate derived from the plan and verified against elapsed wall time, gives a median producer scheduling lag of 1.59 ms for Kafka, not 103 ms — with a *maximum* of 103.5 ms. Two independent instrumentation paths agree: the per-event loop trace and the per-run summary. The 103 ms is real, but it is paid once per run, not once per event.

We cannot call this a repetition of E1's condition, because E1's replay rate is not recoverable (Section 6.5). That is why the argument below does not rest on the comparison between the two campaigns at all. It rests on a sweep that is internally controlled — one campaign, one rate, one match, only the window varying — and on an arithmetic relation that holds at every rate E1 could have used.

A controlled test that separates the two. A median cannot distinguish a per-run cost from a per-event one, because how much of the cost the median sees depends on how many events the run contains — which is precisely what misled us. The quantity that *does* separate them is the *number* of affected events per run. A per-run cost predicts that number is fixed however long the run gets; a per-event constant predicts it grows with the run. We therefore replayed the same match at true real time under three observation windows, holding everything else fixed, and counted events waking more than 50 ms late (Table 20).

The count does not move (Figure 5). Going from the 60 s window to the 600 s window multiplies the events emitted by 8.9 — 57 to 507 — while the number waking more than 50 ms late stays at exactly four and exactly one send blocks, so the share of events paying the cost falls from 7.0% to 0.8%. The median is flat at 1.6 ms throughout and the maximum flat at 103.5 ms: the cost is present in every run and dilutes in every run, which is the signature of something paid once. Had it been a per-event constant, the count would have grown with the window and the median would have stayed at 103 ms.

A once-per-producer cost of this kind is documented upstream: a Kafka producer performs a metadata lookup before its first send to a topic it has not yet cached [2]. We did not isolate that call, so we do not claim it *is* the mechanism — the window sweep constrains the cost's *shape*, not its origin. We note it because the shape we measure and the shape the client documents agree, and because a reader deciding whether this generalises beyond our testbed should know the behaviour is a property of the client rather than of our rig.

Redis was run in the same sweep on the same events, with the same loop trace, and has *no* events waking more than 50 ms late and *no* blocking send at any window. Its worst single event anywhere is under 25 ms. Instrumenting both arms identically matters here for a reason internal to this paper: an earlier version of this table reported Redis's counts as zero when its producer in fact carried no trace, which is the absence of a measurement dressed as one — exactly the failure the paper is about. We added the trace to the Redis producer and re-ran both arms together.

The mechanism, read directly off the loop trace. The per-event trace shows the same five-event signature in every Kafka run, at every window:

event	wake late (ms)	send blocked (ms)
0	0.17	**102.60**
1	**102.96**	0.07
2	**103.08**	0.05
3	**103.17**	0.04
4	**103.30**	0.05
5	0.94	0.27
6	1.42	0.06

The first `produce()` blocks for 102.6 ms fetching metadata and creating the topic. The replay loop is single-threaded, so every event due while it is blocked wakes roughly 103 ms late and then sends in tens of microseconds. There are exactly four of them, and the reason is the sport rather than the harness: this plan's first five events all carry t_{sim}=0, because a kickoff is recorded as a pass, a ball receipt and a carry inside the same second. That is not particular to this match — every one of the eleven plans in our corpus opens with at least five events at t_{sim}=0. A football feed therefore delivers its densest burst precisely when the producer is coldest, which is the worst possible alignment for a start-up cost and the reason it lands in the median rather than the tail. From event 5 the loop is in steady state at about 1 ms. One cost, paid once, is charged to five events; the sixth onward pays nothing. Redis shows no such prologue because `XADD` to a new stream creates it in the same round trip.

That closes it. Five events carry the cost, seven were matched, and the median of seven values of which at least four are 103 ms *is* 103 ms. The published figure is the arithmetic of its own prologue.

Why the original corpus could not see that, and how we know. The E1 runs behind Table 18 replayed the first 600 s of match time, which in this plan holds **507 events**. They matched a **median of seven** — 745 events across 100 Kafka runs, a match rate near one percent. The window was not the problem; the join was. And the seven that survived it are not a random seven.

That last claim is arithmetic rather than inference. Those runs report a median producer scheduling lag of 102.93 ms over seven values, and a median of seven values at 102.93 ms requires *at least four of the seven* to be 102.93 ms or greater. The loop trace says exactly how many events per run are ever that late: four (Table 20), and always the same four — those due while the first send blocks. The matched set must therefore consist almost entirely of the prologue. The plan's first five events are all stamped at t_{sim}=0, so they are the first candidates a consumer sees and the first a lossy join keeps.

The failure thus compounds: a sample of seven is small, but a sample of seven *selected towards the contaminated events* is worse than small. Redis reports 1.75 ms on the same events because opening a stream with `XADD` does not pay what Kafka's first send pays in metadata fetch and topic creation, so the same selection costs it nothing.

Why not knowing E1's replay rate does not matter here. The blocking first send lasts 102.6 ms of *wall* time, so at a replay rate of R times real time it spans $0.103R$ seconds of *match* time. Evaluating that against the plan for each rate E1 could have run at:

Replay rate	Match time spanned	Events inside the prologue
1×	0.10 s	5
120×	12.3 s	16
1200×	123.1 s	112

E1 matched a median of seven events per run. At every one of the three candidate rates the prologue already contains at least four events — enough to carry the median of seven — so the matched sample's median is the start-up cost, which is what the corpus reports. The conclusion is therefore insensitive to the replay rate; Section 6.5 goes on to recover that rate from the same start-up cost, but the withdrawal does not depend on the recovery.

This also explains the three properties we previously offered as evidence, and each of them is equally consistent with a start-up cost: a per-run constant looks *constant* within a run; it is *concurrency-invariant* because every run pays exactly one regardless of N; and it is *rate-dependent* because accelerated replay packs more events into the window and dilutes it. We read three signatures of a per-event mechanism into measurements that a per-run mechanism explains at least as well.

Table 20. The observation-window sweep that separates a per-run cost from a per-event one. Same match, same driver, same broker, N=1, true real-time replay; only the window changes. Both producers are loop-traced, so the counts for the two systems come from the same instrument. Events emitted grow 8.9× across the sweep, but the number of Kafka events waking more than 50 ms late does not move, and exactly one send blocks in every run; Redis has neither at any window. Counts are medians over three replicates from the per-event trace; percentiles come from the per-run summary.

	Window	Events		Scheduling lag (ms)		Events > 50	Blocking
	(s)	emitted	matched	median	max	ms late	sends
Kafka	60	57	57	1.56	103.4	4	1
	180	148	148	1.61	103.5	4	1
	600	507	465	1.58	103.5	4	1
Redis	60	57	57	1.50	17.4	0	0
	180	148	148	1.56	20.4	0	0
	600	507	465	1.53	24.8	0	0

Table 21. Injected one-way broker delay, applied identically to both systems, five feeds. Medians. The zero-delay row is rejected in both arms and marks the absence of a usable baseline.

Delay	Kafka TTI	Kafka transp.	Redis TTI	Redis transp.
0 ms		*rejected*		
5 ms	12.4 ms	5.6 ms	4,651 ms	4,645 ms
20 ms	77.8 ms	20.8 ms	31,401 ms	31,268 ms
50 ms	336.6 ms	44.9 ms	103,143 ms	102,460 ms

The general lesson. The integrity check of Section 4.5 does not catch this. Every one of those runs is causally consistent; nothing is negative; the medians are stable to three significant figures across a hundred runs. What went wrong is that a median over seven events was reported as a property of the system rather than of the window, and the tight agreement across runs made it look robust precisely because the artefact is deterministic. A benchmark can be internally consistent, gate-clean, and still measure its own start-up. We therefore add to Section 8.2 the requirement to report events-per-run alongside any latency percentile, since a percentile over single-digit samples is a statement about the harness.

What we claim instead. On broker transport — the quantity that survived Section 4.5 and is measured over the same events for both systems — Kafka and Redis remain equivalent within one millisecond (Section 7.4). We make no end-to-end claim. The twentyfold figure is withdrawn.

7.6 The condition that reverses the ordering

Injecting one-way delay identically at both brokers with `tc netem` reverses the ordering completely (Table 21). Kafka tracks the injected delay almost additively. Redis collapses: 4.6 seconds at 5 ms of delay, 31.3 at 20 ms and 102.5 at 50 ms, roughly 900, 1,560 and 2,050 times the delay injected. No run was truncated, so this is amplification, not loss. The Redis arm passes the consistency check on all 15 runs at every delay, for the reason given in Section 6.6.

Magnitudes of this size cannot be a per-event cost: 102 seconds cannot be assembled from 50 ms applied once per event. The account we test is a round-trip-bound consumption pattern. Our Redis

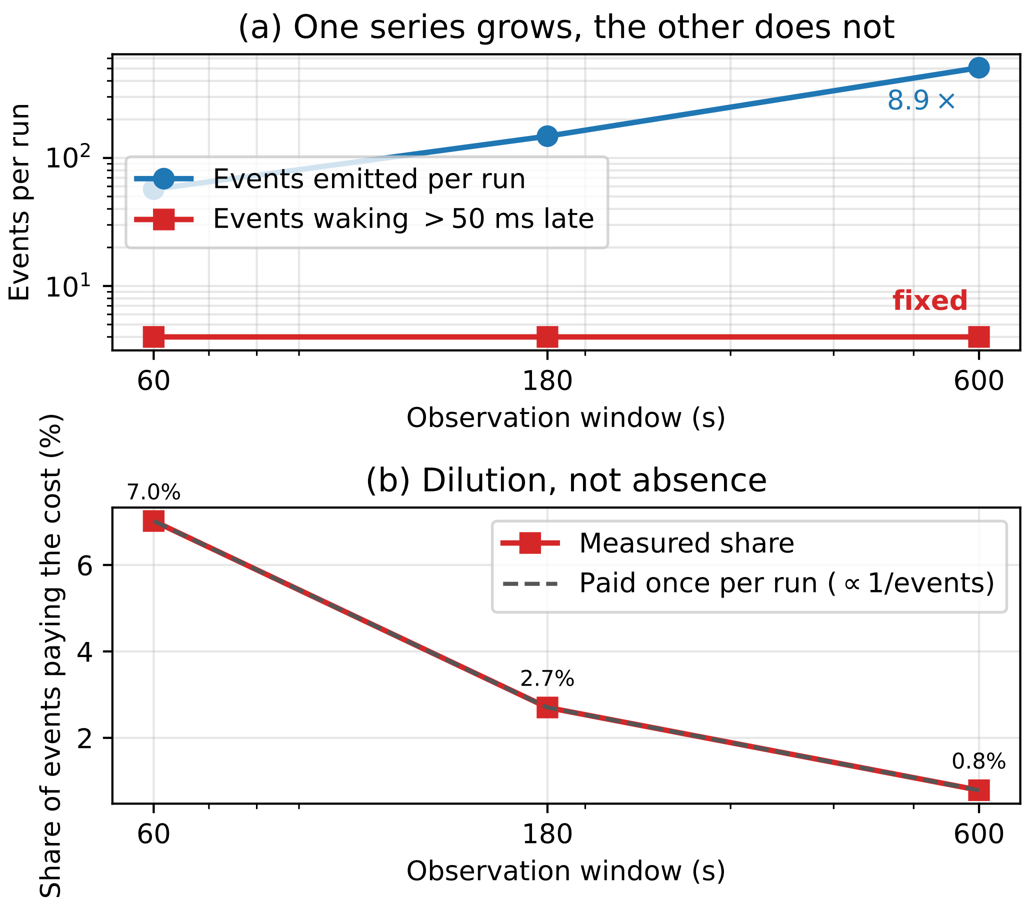

Fig. 5. The window sweep, as a picture. (a) Events emitted per run grow 8.9× while the number waking more than 50 ms late stays at four — a per-event constant would put the two series on parallel lines. (b) The same fact as a proportion: the share of events paying the cost falls as 1/events, the dilution a once-per-run cost predicts. Redis, on the same events, has no affected events at any window.

consumer acknowledges each entry with `XACK` and waits for the reply — the idiomatic pattern in client tutorials — so a read returning two hundred entries is followed by two hundred sequential round trips. On a fast network this is free: a round trip costs 0.22 ms, a ceiling of ≈4,200 exchanges per second against a demand below one. At 20 ms it has no headroom. The queueing statement is elementary [13]: once service rate falls below arrival rate the queue grows without bound and delay is set by the backlog rather than the network. Kafka's consumer serves from a prefetched buffer and commits on a timer, so it has no per-record round trip to expose.

We stated the falsification criterion in advance: if amortising the acknowledgements does not help, the account is wrong. At one feed under true real-time replay, batching acknowledgements in groups of two hundred takes median TTI from 4,138 ms to 103 ms, a factor of **40.2**, replicated four times with a spread of 4 ms unbatched and 0.4 ms batched. Read-loop instrumentation confirms the mechanism and corrects its shape: each `XREADGROUP` returns a *full* batch, a median of 106

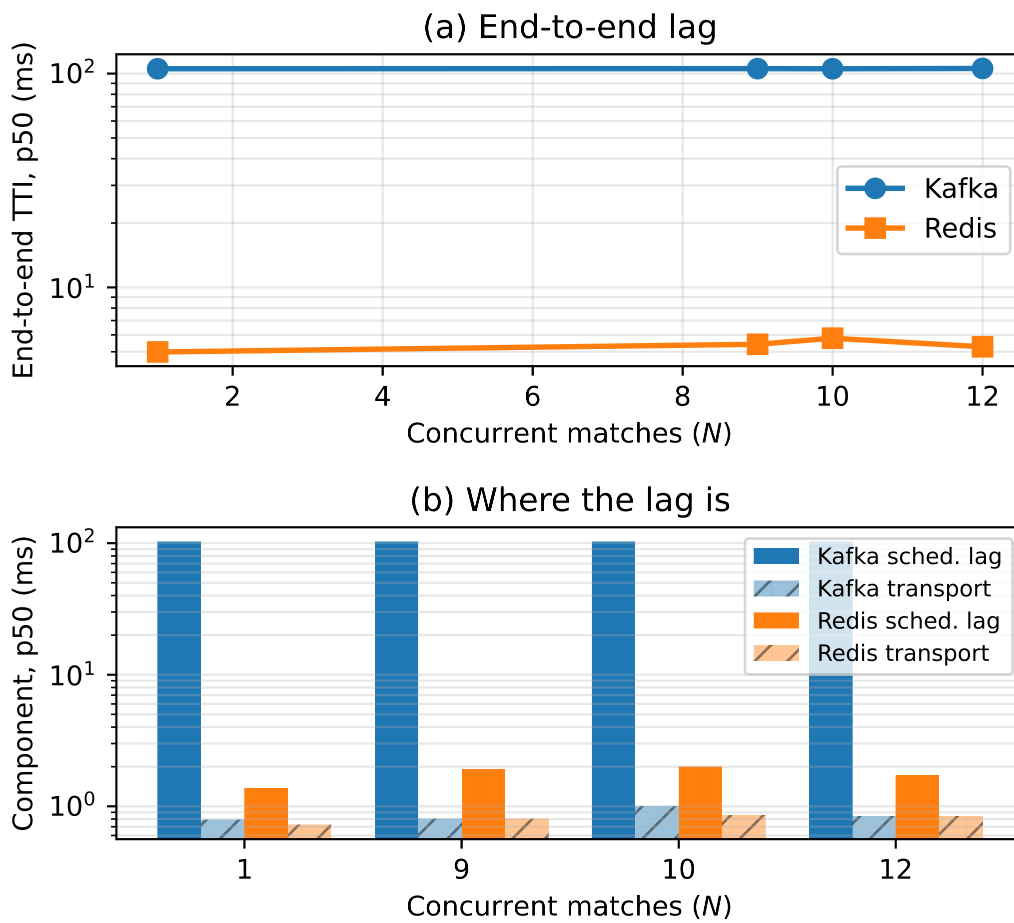

Fig. 6. **Withdrawn.** (a) End-to-end delay against concurrency as originally measured. (b) The decomposition that appeared to place the whole gap in producer scheduling lag. Both panels are retained as evidence for Section 7.5: the runs behind them matched a median of seven events each, so the apparent per-event offset is a per-run start-up cost. Broker transport, the lower series in (b), is unaffected and is what the paper claims.

messages, in about 32 ms, so the consumer is not read-bound. The constraint is entirely in the acknowledgement path.

An unexplained null. At five feeds under accelerated replay the same intervention does nothing: 68,960 → 73,598 ms at 20 ms delay and 87,624 → 92,386 ms at 50 ms, both marginally worse batched. We report this prominently because the obvious explanations do not survive inspection. The treatment was applied — the campaign runs a manipulation check and the consumer logs its effective batch size. The measurement is sound — the Redis arm passes on all 15 runs in each cell. The machine is not saturated in general — Kafka in the same runs sits at 78 ms. An earlier version of this work explained the null away as a consistency failure; that explanation was false and we withdraw it. We claim the mechanism at realistic load, where we have both the intervention and the read-loop evidence, and state that its behaviour at five times that load is unexplained.

7.7 Configuration dominates product

A latency comparison that ignores configuration effort misstates the engineering choice.

Each system has exactly one client setting worth one to two orders of magnitude in delay, and in each case the introductory example in the documentation uses the slow value. For Kafka it is producer pipelining: with one outstanding request per connection, which is what a synchronous send example gives, median TTI at one feed was 1,644 ms; with `max.in.flight` at 64 it was 16 ms, a factor of 103. For Redis it is acknowledgement batching: the per-message `XACK` that every tutorial shows costs 4,138 ms behind a 20 ms hop, and batching two hundred at a time costs 103 ms, a factor of 40.2.

Both share the property that makes them hard to catch: they are *free on a co-located testbed*. Neither matters when the broker round trip is 0.2 ms. A benchmark conducted on loopback prices both at zero and will certify as equivalent two clients that differ by two orders of magnitude in deployment. This is the practical corollary of Section 6.6: the conditions that make a testbed convenient are the conditions that make it least informative.

The systems differ in the *shape* of their configuration surface rather than its size. Kafka's latency-relevant settings are numerous, documented together, and mostly safe once pipelining is enabled. Redis's are fewer, but the consequential one is not a setting at all — it is a property of how the application's read loop is written, which no configuration audit will surface.

8 Discussion

8.1 For benchmark authors

Report a physical-consistency audit. Any delay computed as a difference of timestamps taken in different processes admits a sign check, and the check costs nothing. We rejected 58% of our own runs with it, and every rejected run had passed every conventional check we knew to apply. Better instruments exist [37]; our point is orthogonal to instrument quality — whatever instrument is used, its output should be checked against causality and the retention rate published.

Count your events before you quote a percentile. Our second withdrawal (Section 7.5) came from reporting a median over seven events per run as a property of the system. It survived a hundred runs, a causal-consistency audit, and our own scepticism, because a deterministic per-run artefact reproduces beautifully. Sample size per run belongs next to every percentile, and a benchmark whose window admits single-digit event counts is measuring its own start-up.

Measure at a realistic round-trip time, and treat interventions as interventions. Both configuration defects in Section 7.7 are invisible on loopback, and the one intervention we ran without a manipulation check produced a clean false null.

Make the timestamping path unpreemptable, and say that you did. This is the one mitigation our measurements support directly rather than by inference. Raising the stamping threads to real-time priority, with the background load untouched, cut the inversion rate by 7 to 80× across eight matched pairs and left it at the rate an idle machine shows (Section 7.3). The cost is a scheduling policy on the measurement path only, not a faster clock, a different broker or a quieter machine — and the effect is large enough that a harness which does not do this is measuring its own scheduler along with the system under test. Two caveats keep it honest. The floor is not zero, so this reduces exposure rather than removing it, and the check of Section 4.5 is still needed afterwards. And a real-time stamping thread changes the system it measures; where that is unacceptable, the alternative is to report the retention rate and let a reader judge, not to assume the exposure is absent.

Do not pace the load generator at a rate commensurate with your clock's resolution, and publish the retention rate. This is the mitigation the second failure mode supports directly, and it is the cheapest recommendation in this paper: change a number in a configuration file. A producer paced at a fixed interval that divides evenly into the timestamp quantum puts every sample in a run at the same phase within the quantum, so a run keeps nearly all of its samples or nearly none, and which one is not determined by anything the experimenter set (Section 6.7.3). At 500 msg/s — an interval of exactly 2.000 ms against a millisecond clock, and a rate whose only distinction is that it is round — retention across replicates spanned 99.5 points. At 457 and 383 msg/s it was stable to 2 points.

The remedy is old and belongs to signal processing, where dithering the sampling instant is the standard way to keep a periodic artefact from masquerading as a signal. Applied here it means one of two things: choose a rate incommensurate with the quantum, or, where the rate is fixed by the experiment, add a small random offset to each send so the phase is not shared. Neither costs anything, and either converts an unpredictable retention into a predictable one.

Two things must be published alongside it, because neither is recoverable afterwards. The *retention rate*, for the same reason a survey reports its response rate: without it, a latency summary computed from 0.4% of the samples is typographically identical to one computed from all of them. And the *timestamp resolution beside the measured latency*, because the failure is governed by their ratio — a benchmark reporting whole-millisecond percentiles on a path it believes is sub-millisecond has stated the problem in its own output, if anyone reads the two numbers together.

Do not let three replicates stand in for reproducibility. We ran an identical sweep three times over nine hours. Per-level median retention reproduced at one load level in five, moving by 54 to 98 points at the others (Section 6.7.4). Averaging replicates protects against noise around a stable value; it does nothing when the quantity has no stable value to be noisy around, and the agreement of replicates within one pass is not evidence that it does. Where a quantity may behave this way, the defensible report is the distribution of replicates, not a central estimate of it.

8.2 Applying the check to another benchmark

The check is worth reporting only if it transfers, so we state it as a procedure rather than as a description of what we happened to do. It applies to any latency computed as a difference of timestamps taken in different processes, threads or hosts — which covers most end-to-end measurements in this literature.

(1) **Identify every span whose endpoints are stamped by different execution contexts.** In our pipeline exactly one of the three components qualifies: scheduling lag and the output interval are stamped within a single process, broker transport is not. Spans stamped within one context cannot violate causality and need no check.
(2) **Record the raw per-event values, not summaries.** The failure is invisible in aggregate — that is its defining property — so a pipeline that writes only percentiles has already discarded the evidence. This is the one requirement that may demand a change to an existing harness.
(3) **Count the fraction of events in which the span is negative**, per run and per component.
(4) **Reject by a stated rule, before looking at results.** We reject a run when any component exceeds 1% inverted or has a negative median. The threshold is a judgement and should be reported with its sensitivity (Section 6.3); what matters more is that it is fixed in advance and applied to every condition, including those whose results look reasonable. Applying it only to suspicious results reproduces the error it exists to prevent.
(5) **Report the retention rate alongside the results**, the way a survey reports its response rate. A benchmark that discards nothing has either an unusually clean instrument or an unexamined one, and a reader cannot tell which without the number. Count *both* causes separately: a sample rejected for being negative is an instrument failure, and a sample rejected

for computing to zero is a resolution failure. A single unsigned total cannot distinguish them, and we drew a wrong conclusion from one that could not (Section 6.7.1).

(6) **Compare the timestamp's resolution against the latency you expect to measure, and publish both.** This is the ratio that governs the second failure mode, and neither number alone conveys it. A millisecond-grained clock on a path the experimenter believes is sub-millisecond is a stated problem, not a latent one.

(7) **Read your own reported percentiles for quantisation. This costs nothing and needs no patch.** If every percentile a benchmark prints is a whole multiple of its clock's quantum, the measurement has no resolution below that quantum and the distribution is an artefact of the grid. In the benchmark we audited, all 32 percentile values across eight runs were whole milliseconds, and three runs reported p50 = p95 = p99 = max = 1.0 — a distribution with one value in it, printed to three decimal places, in output anybody could have read. We found it only after instrumenting the source, which was unnecessary.

(8) **Check whether your load generator's send interval is commensurate with that quantum, and change it if it is.** A fixed interval that divides evenly into the timestamp resolution puts every sample in a run at the same phase within it, which makes the surviving fraction bimodal and irreproducible rather than merely small (Section 6.7.3). The diagnostic is arithmetic on two numbers you already know. The remedy is to choose an incommensurate rate, or to dither the send instant where the rate is fixed by the experiment. Note that the rate most likely to be chosen — a round number such as 500 msg/s — is the one most likely to be commensurate with a millisecond clock.

(9) **Report events-per-run alongside every latency percentile.** This is the check that would have caught our second withdrawal (Section 7.5) and the sign check would not: a median over seven events is a statement about the harness, not the system, and it looks its most convincing when the artefact is deterministic enough to repeat across a hundred runs. Any percentile computed over single-digit samples should be labelled as such.

(10) **Record the achieved rate, per run, next to the results — not the flag that was meant to produce it.** A replay harness has a nominal rate and an achieved one, and they part company silently. Ours parted company by a factor of 120 because the plans carried a compression the flag did not know about (Section 6.5). Derive the setting from the workload rather than assuming it, verify the achieved rate against elapsed wall time before the campaign rather than after, and write the verified figure into an artefact that outlives the raw data. We had to reconstruct the rate of our own earliest corpus by arithmetic on its results, which happened to be possible and will not always be.

(11) **Before attributing an offset to every event, vary the run length and count.** Lengthen the observation window by a known factor and count how many events exceed the offset, rather than re-computing the average. A per-run cost holds that count fixed while the run grows; a per-event one grows it in proportion. The two are indistinguishable in any percentile, which is why we report the count. In our sweep the count stayed at four while events per run grew several times over (Table 20).

(12) **Where retention differs between arms, bound the selection it introduces.** Unequal rejection is itself a bias, and Section 6.4 gives the worst-case imputation we use.

The cost is one pass over data the harness already produces. In our implementation the rule is under two hundred lines and runs in seconds over 2,266 runs, which is why we argue it belongs in routine practice rather than in a dedicated study.

When it matters most. Equation 2 tells a reader where to be careful without repeating our experiments: risk rises as the measured quantity approaches the instrument's own scheduling jitter, and as utilisation approaches saturation. A benchmark measuring milliseconds on a loaded machine is in the dangerous region; one measuring seconds, or running well below saturation, is not. That is a rule of thumb, and Section 8.5 says what would be needed to make it quantitative.

8.3 For practitioners

The brokers themselves are equivalent within a millisecond for any workload resembling this one, and neither degrades across its concurrency range. Redis does hold a real, reproducible transport advantage — about 0.4 ms per operation (Section 7.4), the direction the grey literature reports — but at four parts in 100,000 of a seconds-scale end-to-end budget it cannot be the basis for a choice. Selection should rest on durability semantics, operational familiarity and cost — not on a sub-millisecond transport gap, and not on published latency benchmarks conducted at rates the deployment will never see.

Where a consumer sits across a network, audit the *consumption pattern* before choosing a product. A client that acknowledges per message and waits for the reply converts ordinary network delay into seconds of staleness; batching removes it. Kafka is not immune by virtue of being Kafka, but because its client already amortises.

One practical asymmetry survives our withdrawal, and it is a start-up property rather than a steady-state one. A Kafka producer's first `produce()` blocks for roughly 100 ms while it fetches metadata and creates the topic (Section 7.5); Redis's `XADD` creates the stream in the same round trip and blocks for under a millisecond. On a long-lived producer this is paid once and is irrelevant. It matters in two situations: where producers are short-lived — one per match, per request, or per serverless invocation — and where the events that matter most arrive first, as a kickoff burst does. In both, pre-create the topic and issue a warm-up send before the feed opens.

8.4 Threats to validity

Construct validity: the check may not measure what we claim. A negative span is proof that *something* is wrong, but our attribution of it to scheduler delay in reaching the clock is an inference. Two rival explanations are available. Clock skew is ruled out directly for Testbed A, where both processes read the same operating-system clock; it is not ruled out on Testbed B, where producer and consumer may sit on different hosts, and there we rely on the co-located conditions failing at rates comparable to the distributed ones.

Coarse kernel timestamp granularity would also produce inversions, and that rival makes a different prediction we can test. Quantisation acts on each event independently, so its inversions would be scattered at random through a run; a descheduling event, by contrast, makes a *consecutive run* of events wake late together. A Wald–Wolfowitz runs test on the sequence of inversion signs separates the two: independence gives $z \approx 0$, clustering $z \ll 0$. Across conditions the inversions are strongly clustered — median $z = -6.9$ at the co-located floor, -6.8 idle, -5.1 saturated — and the clustering is present even with no background load, so it is intrinsic to the stamping path rather than induced by contention. This is the signature of a shared scheduling delay, not of independent quantisation, and Section 4.1 additionally reports the measured timer granularity on both platforms. We regard the Testbed A attribution as well supported and the Testbed B attribution as probable rather than established.

Internal validity: the audit selects. The check rejects unequally between arms, so the surviving corpus is not a random sample of the measured one. This is the most serious internal threat, and Section 6.4 addresses it with a worst-case bound rather than an argument. That bound holds only while retention exceeds one half, and our tightest cell sits 2.8 points above that line; a slightly worse campaign would have left the equivalence claim undefendable.

Internal validity: the surviving comparison is small. After rejection, per-cell samples run from 8 to 35. The N=1 cell cannot support inference and we treat it as descriptive; conclusions rest on the levels with 19 or more runs.

External validity: one workload, two systems, two platforms. We demonstrate the failure on a sparse bursty feed, two brokers and two operating systems. The model of Section 4.6 predicts it wherever cross-process spans are measured near the instrument's own jitter under load, but we have not shown that. Sharma et al. [29] report the same symptom in an unrelated domain, which is suggestive but is not our evidence.

Construct validity: a percentile is not automatically a property of the system. The 103 ms producer offset reproduced across both testbeds, four concurrency levels and 164 runs, and we took that breadth as evidence it was a property of the system. It was a property of the window (Section 7.5). Reproducibility across runs distinguishes a deterministic effect from a noisy one; it does not distinguish a system effect from a harness effect, and we treated it as though it did. Every percentile in this paper is now reported with the events per run behind it.

Construct validity: the injected-delay sweep is not a clean effect-size manipulation. H1's quantitative slope (Section 7.1) is estimated from a `tc netem` delay sweep, on the assumption that injecting d ms of delay simply raises T_{true} by d. It does not: on our bursty feed, a per-delivery delay builds a queue, so the measured transport *variance* climbs from 10 ms^2 at zero delay to 9,200 ms^2 at 50 ms — a clean offset would leave it unchanged. The delay sweep therefore confounds T_{true} with backlog-driven jitter, and we do not lean on its fitted slope. H1's robust evidence is the comparison it does not confound: the co-located arm, where $T_{\text{true}} \approx 0.5$ ms, fails wholesale, while the network arm, where T_{true} is tens of seconds, passes 15/15 on the same hardware (Section 6.6). Testing the fine-grained slope cleanly needs an instrument that lengthens T_{true} without perturbing the scheduler, which netem is not. The payload sweep of Section 7.3 is that instrument by another route: padding adds serialisation and transfer time without contending for the run queue, and it moves T_{true} by 77× against netem's confounded range. It is not a perfect constant offset either — serialisation pushes utilisation slightly *up*, which works against the observed direction rather than for it — but it is clean enough to carry the claim netem could not, which is why the T_{true} result rests on it and not on this sweep.

The mechanism is established; the quantity is not predicted. The clearest limit on this work is what it still cannot do. We can say what causes an inversion, manipulate it on both sides, and predict the rate to within a third in the three arms where we traced the scheduler directly. We cannot write down the rate a benchmark will suffer at a given load on a given machine. Two attempts failed and are reported above: M/G/1 fits worse than the mean once the sweep reaches the range where forms diverge, and our own registered bracket missed a 1.44× growth by predicting 2.45–3.07×. Both failed for the same reason — both were parameterised in ρ, and ρ is not the variable. The tail-index rule (Equation 5) is the nearest thing we have to a quantitative law, and it carries no load term at all: it predicts how the rate scales with T_{true}, not what the rate is. So the practical guidance this paper supports is a check to run, not a number to look up. A reader wanting to know whether their own measurement is exposed must measure it, which is precisely why we argue the check should be routine rather than the model be trusted.

Conclusion validity: multiplicity and pre-specification. Holm families are declared over the design actually executed rather than chosen afterwards, and equivalence margins were fixed before testing. The threshold in the check was fixed before the final campaign but *after* earlier campaigns had run, which is weaker than pre-registration; the sensitivity analysis of Section 6.3 exists partly to compensate.

8.5 Limitations

The surviving evidence is narrower than the design. The audit removed the throughput sweep, the connection sweep above ten, the cluster arm, the durability quantification and all of Testbed A. We cannot characterise behaviour at the concurrency a multi-tenant platform would see.

E1's replay rate is inferred, not documented. Section 6.5 recovers it from the data and the recovery is, we think, correct, but it is an inference from a 52.34 ms median in one cell rather than a setting anyone recorded. A reader who does not accept the inference should read Section 7.4 as establishing that both arms met the same rate, which is all the between-system comparison needs.

The E1 reconciliation is an explanation, not a re-run of E1. Table 17 shows that restricting the powered runs to their opening seven events reproduces E1's figures in both shift and absolute level, which is why we no longer treat the two campaigns as contradictory. But it demonstrates that the window is *sufficient* to produce the disagreement, not that it is the only difference between the campaigns: E1 ran on a corpus whose replay rate we infer rather than read, and we cannot rule out that some other difference contributes as well. What we can say is that no additional cause needs to be invoked.

The mechanism is established for our client, not for Kafka in general. The blocking first `produce()` is visible in the loop trace of every run (Section 7.5), so for this client the attribution is direct rather than inferred. Whether a different client, or a pre-created topic, or a broker configured against auto-creation would pay the same cost, we did not test. The practitioner advice in Section 8 is written to be safe either way.

The occupancy result is a mechanism, not a load model. Section 7.3 shows by manipulation that the inversion rate follows stamping-thread occupancy rather than utilisation, and that is the strongest causal claim in this paper. It is not a formula. We can say the rate falls fifty-fold when the stamping thread stops being preempted; we cannot say what the rate will be at a given load, and both attempts to write one — M/G/1 and our own bounded bracket — failed against the sweep. A practitioner should read this as "keep the stamping thread runnable and measure what you get", not as a curve to substitute into a budget.

The external evidence is single-host, and the cross-host case is unmeasured. The 80-run campaign of Section 6.7 ran in embedded mode, so both stamps came from one JVM on one host. That is sufficient for the resolution failure, which needs no second clock — the quantum is a property of the timestamp, not of the network — but it means the cross-host channel is established by source audit and not by observation. We attempted a distributed run eight times, at three background-load levels including none at all, and the benchmark's own coordinator-to-worker protocol failed on every one; the load-starvation hypothesis we formed after the first failures is refuted by the unloaded attempts failing identically. That difficulty is worth recording rather than hiding, and it cuts both ways: a failure mode this hard to instrument is one a practitioner is unlikely to find by accident, and a distributed mode that does not complete a run in eight attempts by readers of its source is itself a fact about the state of the tooling.

We are also explicit that the clock bound on this testbed does *not* rule the channel out. `chrony` reports inter-host offsets near 0.1 ms, which is comfortably sub-tick — but the bound it places on its own error, root dispersion plus half the root delay, is 4 to 7 ms per host, so a pair may disagree by up to 12 ms against a 1 ms timestamp. An operator checking whether cross-host subtraction is safe reads the offset line and concludes it cannot happen; the figure that bounds the disagreement is two orders of magnitude larger and three lines further down the same output. The channel is therefore open on this testbed, and we could not measure it.

H3 is measured at one operating point. The symmetric-stamping comparison (Section 7.1) was run at one in-flight request and one load level, because inline stamping requires the single in-flight setting. The direction and the one-sidedness of the effect are what the model predicts, but we do not characterise how the 0.07 ms offset scales with load or pipelining.

The five-feed acknowledgement null is unexplained, as set out in Section 7.6.

One workload. The arrival-rate result should generalise to any sparse bursty feed, but we demonstrate it on one.

9 Conclusion

We set out to compare two message brokers for a real-time sports feed under varying concurrency. We obtained a complete answer, and it was physically impossible.

The audit that found it is this paper's principal contribution. A negative broker delay is proof of instrument failure, and checking for it rejected 1,321 of 2,266 runs, including every run behind a large, significant, theory-confirming result. The rejected corpus had passed plausible medians, narrow intervals, large effect sizes, the architecturally expected direction, and six prior rounds of correction. It failed only against arithmetic. The property that makes the check general is that, being a test against zero, it bites hardest exactly where the measured effect is smallest — which is to say on the delicate comparisons that benchmark papers exist to make.

A second headline then failed the same way, and we withdraw it too. A twentyfold end-to-end gap between the systems, which we had attributed to client behaviour and built a recommendation on, turned out to be a per-run start-up cost read as a per-event constant: the runs behind it matched a median of seven events each. The integrity check does not catch that one. Causal consistency is necessary and not sufficient, and a percentile computed over single-digit samples describes the harness rather than the system — the more so because a deterministic artefact reproduces across a hundred runs and looks robust for exactly the wrong reason.

What survives is narrower and better evidenced. The brokers are equivalent within a millisecond and neither degrades with concurrency, robustly to the audit's own selection effect — though a powered replication resolves inside that margin a real 0.41 ms advantage for Redis that the original seven-event corpus reported as a tie. Each system has one client setting worth one to two orders of magnitude, both free on the co-located testbeds where such settings are normally evaluated. And the model of the failure now carries measured rules rather than a derivation: inversion probability falls as the measured quantity grows, rises with concurrent process count, and rises monotonically with utilisation; and a symmetric instrument removes a quarter of the residual between-system difference, the same asymmetry that manufactured our first result in the large. We had also claimed the M/G/1 form specifically, and withdraw it — not as undecided but as refuted. Extending the sweep to the utilisation range where the candidate forms actually diverge, M/G/1 fits *worse than the mean* ($R^2 = -0.05$ against a fitted exponential's 0.93). Our own bounded prediction failed on the same data. What was an anecdote at the start of this work is now a mechanism, and the curve we thought we were fitting turned out not to be the thing that matters.

A dedicated campaign then corrected the model we had just confirmed. Treating the stamping asymmetry Δ as a single distribution is only a leading-order account: it is a mixture of a narrow jitter core and a rare tail from descheduling, and load multiplies the tail's weight roughly twelve times faster than it widens the core. We pre-registered that test expecting the simpler form to fail, and it did. The practical reading is less comfortable than the tidy version: a benchmark can sit at a load where its numbers look tight, because the core is narrow, while the tail that will corrupt them is already growing underneath.

The fact that forced a mechanism rather than a curve was the clustering. Inversions arrive in runs of consecutive events at every load, idle included, and no distribution predicts that — a distribution says how large a delay is, never which events receive one. What does predict it is a thread that is periodically *not running*: being preempted is an interval, so the events inside it are corrupted together. On that reading the mixture is not two distributions but two states, load sets how often the instrument is in the bad one, and the tail curves at different loads should be parallel rather than rescaled — which is what we find, a median factor of 1.34 against the scale family's 23. We reached this by looking at our data and say so, and then ran the confirmatory tests we had pre-registered.

They hold, and they decide the mechanism by manipulation rather than by fit. Raising the stamping threads to real-time priority at *fixed* utilisation collapses the inversion rate by 7 to 80× across eight matched pairs. Two loads reaching *identical* utilisation by different geometries — cores left free against every core duty-cycled — differ by 2.07×, and a function of ρ cannot return two values for one input, so ρ is not the variable. Lengthening the true transport 77× *lowers* the rate, the sign no account based on system stress predicts. And the occupancy we had been forced to infer is now measured directly: a kernel trace of run-queue delay predicts the observed inversion rate to within a third, in three arms at two load levels, with nothing fitted between them — and, in the arm where the rate is smallest, the tracer removes the effect it came to observe, which is this paper's own argument arriving uninvited. The stall distribution's tail index is near 0.34, below one, so it has no finite mean — which is why the cumulative counters we tried first could not account for the effect, and why an instrument built on averages is structurally rather than accidentally blind to it. The advice this implies is not a better clock but a stamping thread the workload cannot preempt.

A paper that only ever audits its own mistakes proves that its authors are careless, not that its method is needed. So we turned the argument outward, and found a second failure mode that is not ours and not the one we went looking for. The OpenMessaging Benchmark [24] computes end-to-end latency as a consumer-side clock read minus a timestamp written on the producer host, and records the sample only if it is positive, counting nothing that it drops. We instrumented that guard and ran the benchmark 80 times.

What we expected to find, we did not. We had reported that a single such run discarded 6,000 samples under load, and read those as the same causality violation we report above. The counter behind that reading could not distinguish sign. Separated across all 80 runs, roughly 420,000 discarded samples contain *not one negative*. We withdraw the claim, and record where its refutation was: in our own committed artefact, in a field we had printed and never read. What was missing was not data but a reason to look at the sign, and the statistic we had chosen to report did not have one.

What is there instead is worse for the benchmark and better evidenced. It computes its reported latency distribution from between 0.36% and 100% of the samples it takes, and reports a median of 1.0 or 2.0 ms in every case — a headline insensitive to a 278-fold change in the evidence behind it, beside a retention rate that is unreported and unrecoverable from a completed run. Which case obtains is set by a variable no operator chooses: whether the producer's send interval is commensurate with the timestamp resolution. We show this by manipulation. At 500 msg/s, an interval of exactly 2.000 ms, retention across replicates spans 99.5 points; at 457 and 383 msg/s it is stable to two, and those two arms agree with *each other* despite intervals differing by 19% — which a dependence on the interval could not produce and a dependence on phase requires. The standard defence fails as well: a three-replicate median reproduced at one load level in five across three passes of an identical configuration, because the quantity has no central value to converge on.

So the two failure modes are one phenomenon seen twice. Both compare a timescale belonging to the instrument against the interval under measurement — a scheduling stall in the first case, a clock tick in the second — and both therefore worsen as that interval shrinks. A benchmark on a sub-millisecond path is exposed to both at once, and neither is visible in what it prints. We claim no published result is wrong, because we have audited no deployment but our own. What we know is that both failures are reachable, large, and unreported in software nobody in this study wrote.

One smaller episode makes the same point from the other side. Checking our own provenance, we found that no surviving artefact recorded the replay rate of the earliest corpus, and that the records we did have disagreed. We recovered it anyway — the start-up cost is a wall-clock constant, so the number of events it swallows encodes the rate, and one cell of fifteen runs reading 52.34 ms where both modes are 1.6 and 103.5 fixes it at four late events and hence at true real time. That is the same instrument as the sign check: a physical constraint deciding what the bookkeeping could not. It is also luck, and no basis for a method. The rule is to write the number down.

We began by asking which of two brokers is faster for a football feed. The honest answer is that it barely matters — Redis holds a reproducible 0.41 ms advantage that is real, and negligible beside the seconds a human annotator takes to enter the event. That question turned out to be the least interesting thing we could have asked.

What replaced it is a condition rather than a comparison. When the interval you are measuring approaches a timescale belonging to your instrument, the measurement fails in ways the measurement cannot report: it goes negative, or it disappears, and in both cases the summary that reaches the reader is plausible, stable, and wrong. That condition is not exotic. It is the ordinary operating point of every co-located broker benchmark, because the whole point of such a benchmark is that the path is fast — and it is precisely on the small differences those benchmarks exist to resolve that both failures bite hardest.

We report the road in full, including three withdrawals and a model we had to correct, because the path ran entirely through measurements we had every reason to trust, and because what settled each of them was never a better clock or a larger sample. It was a constraint the data could not violate: a latency cannot be negative, and a benchmark cannot summarise samples it has thrown away. Both are free to check. Neither was being checked.

Artefact Availability

All code, replay plans, aggregated results and analysis are available at https://github.com/Gustolandia/streaming-latency-sports. Event data is the StatsBomb open dataset under CC BY-NC 4.0; raw files are not redistributed but are re-fetchable from a pinned upstream commit. Every run records its git commit and per-file SHA-256. The plots are regenerated from committed CSVs by scripts in the repository, and a consistency suite recomputes the manuscript's quoted quantities from those CSVs and fails if paper and data disagree. We state its reach exactly, because a claim of total coverage would be the kind of claim this paper is about: the suite checks the values it names, which include every headline number, every table this paper argues from, and the sample size behind each; it does not enumerate every digit in every table. A companion script reports which table cells rest on a recomputing test rather than on a value stored verbatim in an artefact, so the boundary is visible rather than assumed.

References

[1] Apache Kafka. 2015. KIP-32: Add timestamps to Kafka message. Apache Software Foundation, Kafka Improvement Proposal. Defines CreateTime (timestamp set by the producer) and LogAppendTime (overwritten by the broker); CreateTime is the default.

[2] Apache Kafka. 2020. KIP-526: Reduce producer metadata lookups for large number of topics. Apache Software Foundation, Kafka Improvement Proposal. Documents the metadata fetch a producer performs before its first send to an uncached topic.

[3] Arvind Arasu, Mitch Cherniack, Eduardo Galvez, David Maier, Anurag S. Maskey, Esther Ryvkina, Michael Stonebraker, and Richard Tibbetts. 2004. Linear Road: A Stream Data Management Benchmark. In *Proceedings of the 30th International Conference on Very Large Data Bases (VLDB)*. VLDB Endowment, Toronto, Canada, 480–491.

[4] Ashok Chandrasekar and Jason Kramberger. 2026. Identifying and mitigating systemic measurement bias in production LLM inference benchmarks. *arXiv preprint arXiv:2605.24217* (2026).
[5] James C. Corbett, Jeffrey Dean, Michael Epstein, Andrew Fikes, Christopher Frost, J. J. Furman, Sanjay Ghemawat, Andrey Gubarev, Christopher Heiser, Peter Hochschild, et al. 2013. Spanner: Google's globally distributed database. *ACM Transactions on Computer Systems* 31, 3 (2013), 1–22.
[6] ThreadSafe Diaries. 2025. I Benchmarked Kafka, RabbitMQ, and Redis Streams, The Winner Surprised Me. Accessed: June 15, 2026. https://medium.com/@ThreadSafeDiaries/i-benchmarked-kafka-rabbitmq-and-redis-streams-the-winner-surprised-me-cf3f484eb7b2
[7] Bradley Efron. 1979. Bootstrap methods: another look at the jackknife. *The Annals of Statistics* 7, 1 (1979), 1–26.
[8] Guenter Hesse, Christoph Matthies, Michael Perscheid, Matthias Uflacker, and Hasso Plattner. 2021. ESPBench: The Enterprise Stream Processing Benchmark. In *Proceedings of the ACM/SPEC International Conference on Performance Engineering*. ACM, Virtual Event, France, 201–212.
[9] J. L. Hodges and E. L. Lehmann. 1963. Estimates of location based on rank tests. *The Annals of Mathematical Statistics* 34, 2 (1963), 598–611.
[10] Sture Holm. 1979. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics* 6, 2 (1979), 65–70.
[11] Raj Jain. 1991. *The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling*. Wiley, New York, NY, USA.
[12] JusDB. 2025. Redis Streams vs Kafka: Event Streaming Architecture Comparison 2025. Accessed: June 15, 2026. https://www.jusdb.com/blog/redis-streams-vs-kafka-event-streaming-comparison-2025
[13] Leonard Kleinrock. 1975. *Queueing Systems: Volume I - Theory*. Wiley-Interscience.
[14] Jay Kreps, Neha Narkhede, and Jun Rao. 2011. Kafka: a distributed messaging system for log processing. *Proceedings of the 6th ACM Symposium on Networked Systems Design and Implementation* (2011).
[15] William H. Kruskal and W. Allen Wallis. 1952. Use of ranks in one-criterion variance analysis. *J. Amer. Statist. Assoc.* 47, 260 (1952), 583–621.
[16] Daniël Lakens. 2017. Equivalence tests: a practical primer for t tests, correlations, and meta-analyses. *Social Psychological and Personality Science* 8, 4 (2017), 355–362.
[17] Leslie Lamport. 1978. Time, clocks, and the ordering of events in a distributed system. *Commun. ACM* 21, 7 (1978), 558–565.
[18] Jialin Li, Naveen Kr. Sharma, Dan R. K. Ports, and Steven D. Gribble. 2014. Tales of the tail: Hardware, OS, and application-level sources of tail latency. In *Proceedings of the ACM Symposium on Cloud Computing (SoCC '14)*. ACM, Seattle, WA, USA. doi:10.1145/2670979.2670988
[19] Hongyou Liu, Will Hopkins, A. Miguel Gomez, and J. Sampaio Molinuevo. 2013. Inter-operator reliability of live football match statistics from OPTA Sportsdata. *International Journal of Performance Analysis in Sport* 13, 3 (2013), 803–821.
[20] Jean-Pierre Lozi, Baptiste Lepers, Justin Funston, Fabien Gaud, Vivien Quéma, and Alexandra Fedorova. 2016. The Linux Scheduler: a Decade of Wasted Cores. In *Proceedings of the Eleventh European Conference on Computer Systems (EuroSys)*. ACM, London, United Kingdom. doi:10.1145/2901318.2901326
[21] Henry B. Mann and Donald R. Whitney. 1947. On a test of whether one of two random variables is stochastically larger than the other. *The Annals of Mathematical Statistics* 18, 1 (1947), 50–60.
[22] David L. Mills. 1991. Internet time synchronization: the network time protocol. *IEEE Transactions on Communications* 39, 10 (1991), 1482–1493.
[23] Muzeeb Mohammad. 2025. Analysis of design patterns and benchmark practices in Apache Kafka event-streaming systems. *arXiv preprint arXiv:2512.16146* (2025).
[24] OpenMessaging Project, Linux Foundation. 2018. The OpenMessaging Benchmark Framework. Open standard, multi-platform messaging performance benchmark. Available at openmessaging.cloud/docs/benchmarks/.
[25] Luca Pappalardo, Paolo Cintia, Alessio Rossi, Emanuele Massucco, Paolo Ferragina, Dino Pedreschi, and Fosca Giannotti. 2019. A public data set of spatio-temporal match events in soccer competitions. *Scientific Data* 6, 1 (2019), 236.
[26] Salvatore Sanfilippo. 2017. Redis streams: a new data type for real-time streaming. *Redis Labs Blog* (2017). https://redis.io/topics/streams-intro
[27] Bianca Schroeder, Adam Wierman, and Mor Harchol-Balter. 2006. Open versus closed: a cautionary tale. In *Proceedings of the 3rd USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. ACM, Pisa, Italy, 239–252.
[28] Donald J. Schuirmann. 1987. A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability. *Journal of Pharmacokinetics and Biopharmaceutics* 15, 6 (1987), 657–680.
[29] Ankur Sharma, Deep Shah, David Lariviere, and Hesham ElBakoury. 2026. Time, Causality, and Observability Failures in Distributed AI Inference Systems. *arXiv preprint arXiv:2604.21361* (April 2026). Open Compute Project, Unified Intelligent Infrastructure workstream.

[30] Anshu Shukla, Shilpa Chaturvedi, and Yogesh Simmhan. 2017. RIoTBench: An IoT benchmark for distributed stream processing systems. *Concurrency and Computation: Practice and Experience* 29, 21 (2017), e4257.

[31] StatsBomb. 2023. StatsBomb Open Data. *GitHub Repository* (2023). https://github.com/statsbomb/open-data

[32] Michael Stonebraker, Ugur Cetintemel, and Stan Zdonik. 2005. The 8 requirements of real-time stream processing. *ACM SIGMOD Record* 34, 4 (2005), 42–47.

[33] Akul Swami and Nikhil Chougule. 2026. Architecture dependent temporal observability under deployment interference in edge inference systems. *arXiv preprint arXiv:2605.17701* (2026).

[34] Gil Tene. 2015. HdrHistogram: A High Dynamic Range Histogram. Records values with configurable precision and rejects negative values. Ships an explicit correction for coordinated omission (recordValueWithExpectedInterval) and none for resolution-driven discard. Used by the OpenMessaging Benchmark. Available at hdrhistogram.org.

[35] Gil Tene. 2015. How NOT to Measure Latency. Coined "coordinated omission": a load generator whose measurement thread blocks during a stall never takes the samples that would have been worst, so reported percentiles are optimistic by orders of magnitude. The failure reported in Section 6.7 is its mirror image — the samples are taken and then discarded downstream. Talk, Strange Loop / QCon..

[36] Pete Tucker, Kristin Tufte, Vassilis Papadimos, and David Maier. 2008. *NEXMark: A Benchmark for Queries over Data Streams*. Technical Report. OGI School of Science and Engineering, Oregon Health and Science University.

[37] Shinhyung Yang, Jiun Jeong, Bernhard Scholz, and Bernd Burgstaller. 2022. Cloudprofiler: TSC-based inter-node profiling and high-throughput data ingestion for cloud streaming workloads. *arXiv preprint arXiv:2205.09325* (2022).

[38] Praneeth Yerrapragada. 2025. kafka-bullmq-benchmark: A comprehensive distributed systems performance comparison. Includes white paper with methodology; Accessed: June 15, 2026. https://github.com/praneethys/kafka-bullmq-benchmark